Enterprise WhatsApp platform architecture

A technical blueprint for a product with capabilities similar to WhatChimp

Version 2 • Enterprise design from day one • 29 September 2026

Build a multi-tenant messaging platform around Meta’s official WhatsApp Business Platform, with regional isolation, durable message processing, a visual automation builder, a shared agent inbox, campaigns, integrations, agency workspaces, ad attribution, agent tools and controlled AI assistance. The central design principle is to persist intent before executing an external action and preserve enough evidence to recover safely after failure.

Architecture decision

Use bounded regional cells, each spanning three availability zones. A tenant has one home cell and one active write region. Deploy a separate control plane for tenant provisioning, subscriptions, and placement. Keep message processing operational with a cached, signed configuration when the control plane is unavailable. Provide dedicated cells for customers requiring stronger infrastructure isolation.

What this document provides

The following pages connect product capabilities to network routes, services, data stores, events, failure handling, and operational controls. Architecture and flow diagrams show how onboarding, inbound messages, outgoing replies, campaigns, bot execution, AI, and disaster recovery work. API examples, capacity calculations, implementation phases, and acceptance tests make the design actionable for an engineering team.

Scope and assumptions

This is a proposed architecture for a similar product, not a description of WhatChimp’s private implementation. AWS is the concrete infrastructure reference; the boundaries are portable to other clouds. WhatsApp is the primary channel, with adapters for Instagram, Messenger, and web chat. Voice calling and payment-card processing are separate future domains. All capacities and SLOs are design targets to validate under load, not vendor guarantees.

The commercial model assumes tenant-owned WhatsApp accounts and, where available, direct provider billing. Partner eligibility, permissions, rates, throughput entitlements, and regional availability must be confirmed for the chosen launch markets and Meta API version. Source notes and official references appear at the end of this document.

Document map

Version 2 integrates the review corrections into the enterprise design. The revision map links the reported gaps to their resolution. Use the linked contents or PDF bookmarks to navigate.

Enterprise launch scope and revision map 3
1 Product capabilities and system context 4
2 Network topology and trust boundaries 5
3 Connection and protocol inventory 6
4 Regional cells and tenant placement 7
5 Regional ingress acknowledgment contract 8
6 Agency organizations and delegated access 9
7 Service boundaries and ownership 10
8 Tenant onboarding and channel connection 11
9 Inbound webhook pipeline 12
10 Ingress keys and conversation ordering 13
11 Outbound message pipeline 14
12 Dispatch outcomes and reconciliation 15
13 Message capabilities and interaction context 16
14 Events ordering and reliable state changes 17
15 Core data model 18
16 Storage constraints and retention 19
17 Contact import and identity validation 20
18 Tags segments and safe bulk operations 21
19 Consent templates and send policy 22
20 Account health and scoped admission state 23
21 Management webhooks and provider contracts 24
22 Campaign and sequence pipeline 25
23 Ad referrals attribution and conversion events 26
24 Visual chatbot builder and publication 27
25 Durable automation runtime 28
26 Shared inbox and realtime collaboration 29
27 Agent productivity SLA timers and push 30
28 AI knowledge and response pipeline 31
29 Integration media and commerce pipelines 32
30 Security and tenant isolation 33
31 Public API and event contracts 34
32 Enterprise command and storage contracts 35
33 Analytics billing and audit pipelines 36
34 Availability targets and observability 37
35 Capacity model and cost drivers 38
36 Enterprise capacity and isolation budgets 39
37 Failure handling and disaster recovery 40
38 Delivery pipeline and deployment 41
39 Verification and launch acceptance 42
40 Review regression tests and evidence 43
41 Enterprise implementation sequence 44
42 Decisions to finalize before engineering 45
Sources and implementation references 46
Further references and validation notes 47
Additional provider and notification references 48

Notation: Every diagram is a logical view with explicit actors and data directions. Protocol and trust details appear in the connection inventory. Tenant means a customer workspace; cell means its bounded deployment; provider portfolio and agency organization are separate identities.

Enterprise launch scope and revision map

Version 2 retains regional cells, three-zone operation, Kafka, durable workflows, isolated tenant storage, controlled outbound dispatch, and tested regional recovery. These are launch foundations. Product delivery is staged internally, but general availability requires the enterprise acceptance gates in this document.

Review finding	Resolution in this edition
Network diagram omitted paths	Separate application workers from managed data infrastructure; draw outbox publication and the complete external egress path.
Receipt callback drawn from adapter	Draw Meta as the webhook producer; add independent status ingestion and all dispatch outcome branches.
Ingress key and conversation key unclear	Define account/shard ingress, participant routing, transactional conversation resolution, and conversation-event publication.
Cross-region acknowledgment ambiguous	Specify a per-request durable acceptance barrier, partial-success retries, unknown mappings, and residency boundaries.
Channel health insufficiently modeled	Add provider portfolio, account, number and template observations, scoped capacity, freshness rules, and management-event adapters.
Missing product operations	Add imports, saved segments, tags, bulk jobs, ad referral attribution, rich-message capabilities and agent productivity.
Flat tenant model	Add agency organizations, child workspaces, explicit delegated grants, billing rollups and verified brand domains.
Agent notification gaps	Add persisted SLA deadlines, notification intents, device registry, APNs/FCM delivery and authenticated resynchronization.
Scope estimate too confident	Replace the calendar promise with evidence-based release gates, capacity validation and staffed service ownership.

Meaning of enterprise from day one

No launch tenant depends on an in-memory queue, untested restore, shared administrator credential, or unbounded campaign fan-out. Cell placement, home-region ownership, per-tenant quotas, role enforcement, audit evidence, deletion, recovery and operational coverage exist before external production onboarding. Additional cells are provisioned from the same tested infrastructure definition as demand grows.

Provider-specific names in the management adapter registry are explicitly marked for live verification. Platform-owned contracts and their failure behavior are specified independently of that verification; account activation fails its readiness check if required subscriptions, scopes or payload fixtures are missing.

1 Product capabilities and system context

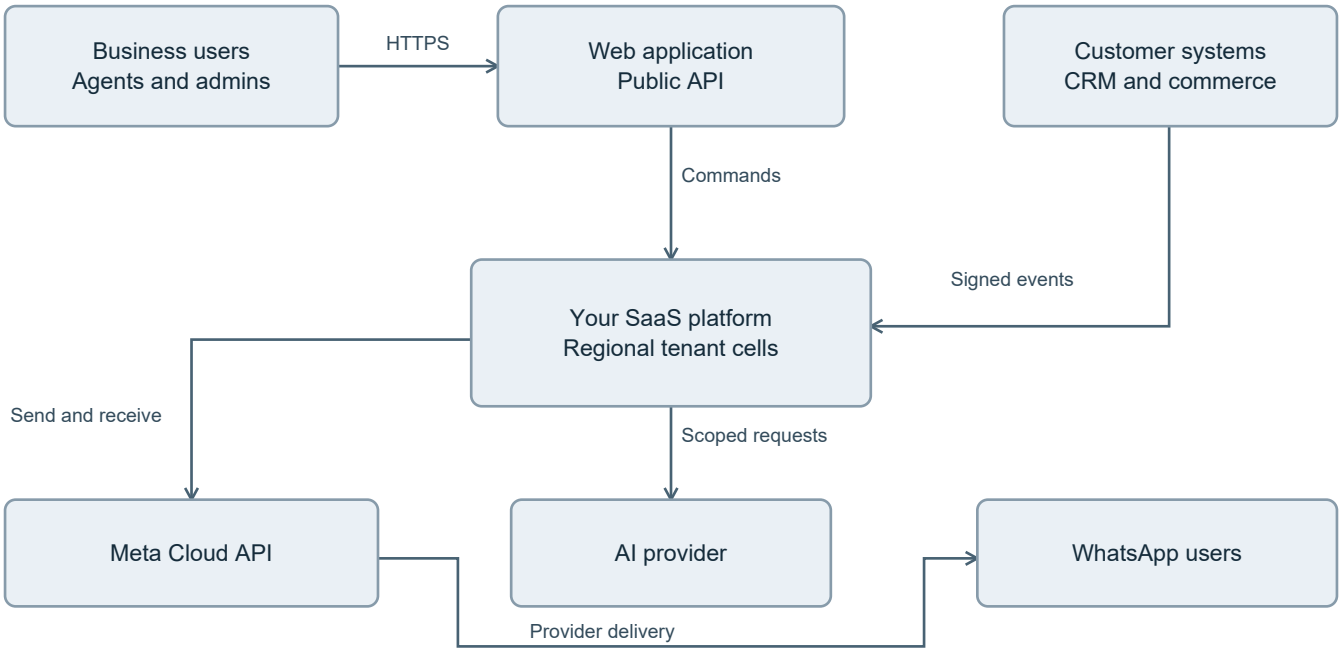


Figure 1. The platform owns business workflows and records; Meta owns delivery over the WhatsApp network. The AI provider is optional in the reply path.

Capability	Services that make it work
Team inbox	Conversation ownership, agent routing, realtime fan-out, message commands
Broadcasts and sequences	Audience snapshots, policy evaluation, scheduling, quotas, outbound dispatch
Visual bots and AI	Flow compiler, durable runtime, knowledge retrieval, tool authorization
CRM and commerce	Contact identity, custom fields, connectors, media, catalogs and webhook delivery
Enterprise administration	SSO, RBAC, audit, tenant placement, billing, retention and exports

WhatChimp publicly describes campaigns, shared inboxes, chatbot flows, AI agents, and integrations. These define the feature reference, while the service decomposition here is a new engineering proposal. [1, 2]

2 Network topology and trust boundaries

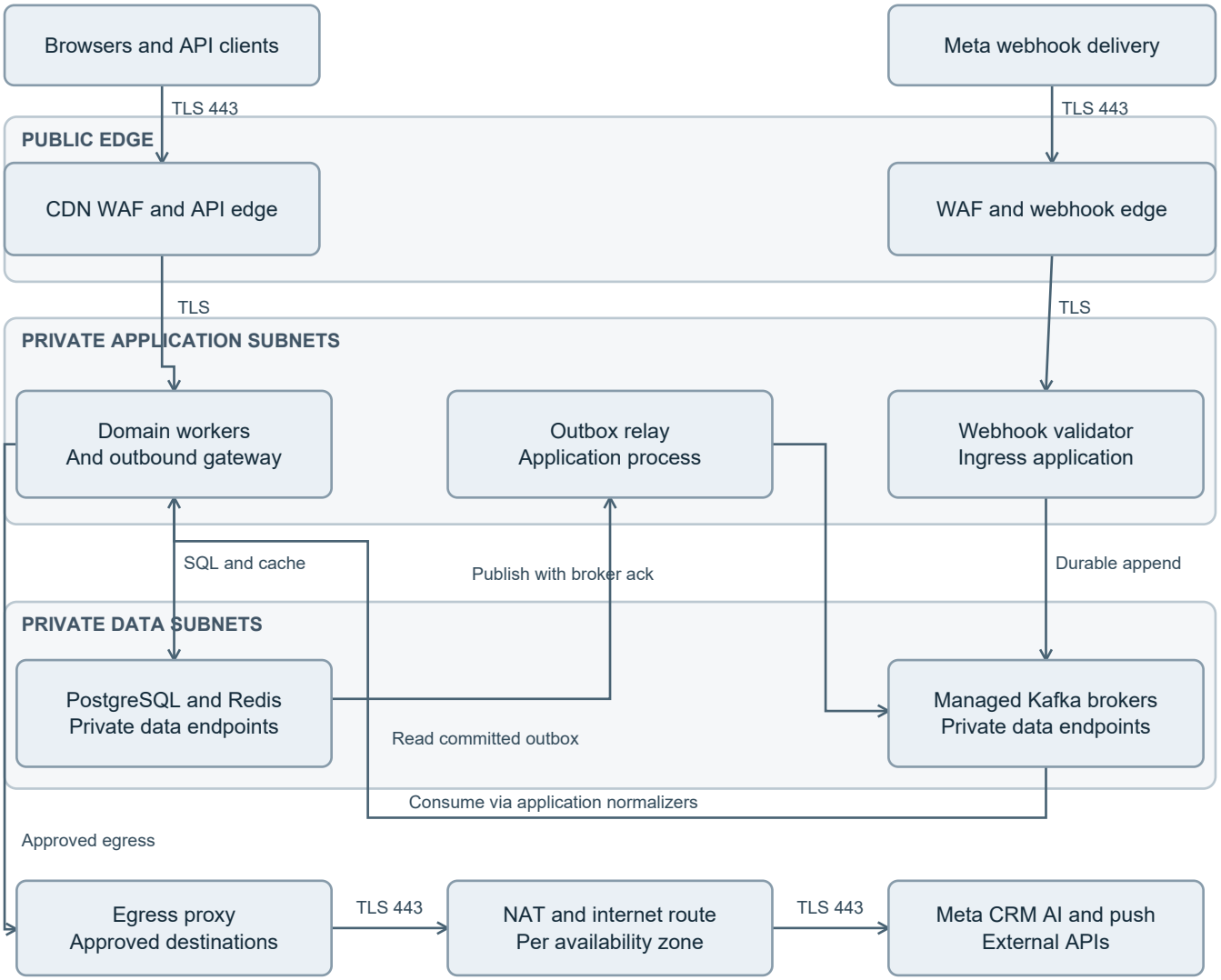


Figure 2. Logical data paths across separate trust boundaries. Consumers initiate broker connections; arrows for consumed events show data direction. External callbacks enter through the webhook edge independently of outbound API calls.

Repeat the private deployment across three availability zones. Managed broker endpoints and database endpoints reside in the data boundary; normalizers and outbox relays run as application workloads. Domain writes and outbox rows commit together. The relay reads committed rows, publishes with acknowledgments, and safely retries; duplicate publication is handled by consumers.

Use separate production, non-production, security and recovery cloud accounts. No application pod or database has a public IP. Object storage and secrets are reached through private service endpoints, omitted above for clarity. Approved external calls use a proxy plus zone-local NAT and internet gateway; replies return on the established connection. Default-deny policies explicitly allow DNS and approved service identities. [11]

3 Connection and protocol inventory

Initiator and destination	Protocol	Authorization and purpose
Browser → CDN / API edge	HTTPS 443	OIDC session cookie; CSRF protection for changes. Static assets may be public; tenant responses are never shared-cache content.
Browser → realtime gateway	WSS 443	Short-lived, audience-bound connection token; tenant and conversation access checked on subscribe.
Meta → webhook endpoint	HTTPS 443	GET challenge token for setup; POST body HMAC with app secret. Preserve raw bytes until verification.
Edge → gateway / ingress	TLS 443	Origin access control plus service identity; no direct public bypass.
Service → service	mTLS 8443	Workload identity, explicit allowed caller, request deadline, tenant context and trace ID.
Services → PostgreSQL pool	TLS 5432	Per-service role; transaction-scoped tenant context; bounded connection pool.
Services → managed Redis	TLS 6379	Named ACL user; cache, presence and rate counters. Never the only message record.
Services → Kafka brokers	TLS 9094	Concrete reference listener; workload auth and topic ACLs. Port changes with provider authentication mode.
Workers → workflow engine	TLS 7233	Private Temporal frontend; namespace permissions and payload encryption.
Workers → cloud services	HTTPS 443	Private endpoints and workload IAM roles; tenant-scoped object keys.
Egress → Meta / CRM / AI	HTTPS 443	Server-held provider credentials; bounded timeout and destination policy.
Operator → admin plane	HTTPS 443	SSO, MFA, device posture and time-limited role. Database access only through audited private access.
Pods → approved DNS	UDP / TCP 53	Cluster resolver only. Block arbitrary DNS and metadata endpoints.
Services → telemetry collector	mTLS 4317	OTLP telemetry with redaction and workload identity; collector exports over TLS.

All entries are proposed allowed paths. Deny other ingress and egress by default. Set TLS certificate rotation, request/body size limits, connection budgets, and idle timeouts per endpoint. WAF rules for webhooks must not require browser challenges. A provider source-IP allowlist may supplement signature validation only if supported and maintained; it cannot replace it. [6, 11]

Additional enterprise paths: Routing ingress → home ingress uses mTLS over 443 with a signed tenant/placement envelope and a bounded request deadline. Notification workers → APNs or FCM use authenticated HTTPS 443 through the egress proxy. Native agent apps → API use HTTPS 443 with OAuth/OIDC sessions; push payloads grant no access. Kafka consumers and outbox producers each receive separate topic ACLs.

4 Regional cells and tenant placement

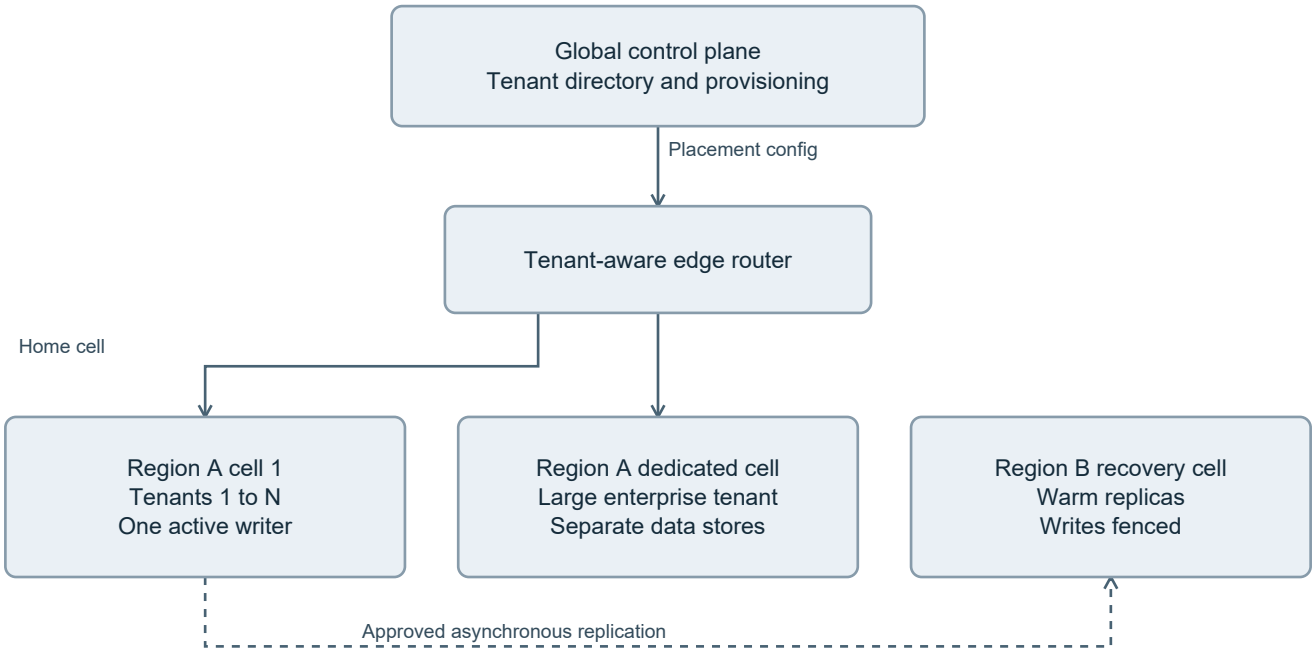


Figure 3. A tenant is active in one region at a time. Other tenants may be active in different regions; a global multi-master message database is not required.

Routing and availability

The directory maps provider app, portfolio, WABA and phone-number identifiers to authorized tenant bindings, home_region, cell_id and placement_epoch. Management events can lack a phone-number ID and use their account or portfolio binding. Gateways cache signed snapshots. Validate account mapping after webhook signature verification. Use a region-specific webhook route where the provider configuration supports it; otherwise a minimal routing ingress must forward each item to its home region only through authorized processing regions. The Regional ingress acknowledgment contract specifies durable acknowledgment, forwarding failures and retries.

Run at least three replicas for ingress and critical stateless services, spread across zones. Use anti-affinity, disruption budgets, readiness probes, and connection draining. Every cell has its own database cluster, event broker allocation, workflow namespace, quotas, and credentials boundary. Size the cell for the loss of one zone, not just average load.

Isolation and migration

Start with several bounded cells rather than one shared global database. Dedicated-cell placement is a provisioning option, not a fork of the software. Move a tenant by quiescing new writes, draining its work, copying data and dedupe history, checking counts and checksums, advancing the placement epoch, and resuming only in the target. Old workers must reject a newer epoch to prevent split-brain sends.

Select regions from customer residency contracts, provider availability, latency tests, and operational coverage. Cross-region backups, AI inference, telemetry, support access, and routing metadata all need the same placement policy. A tenant prohibited from cross-border replication needs an approved local recovery design with a different disaster profile.

5 Regional ingress acknowledgment contract

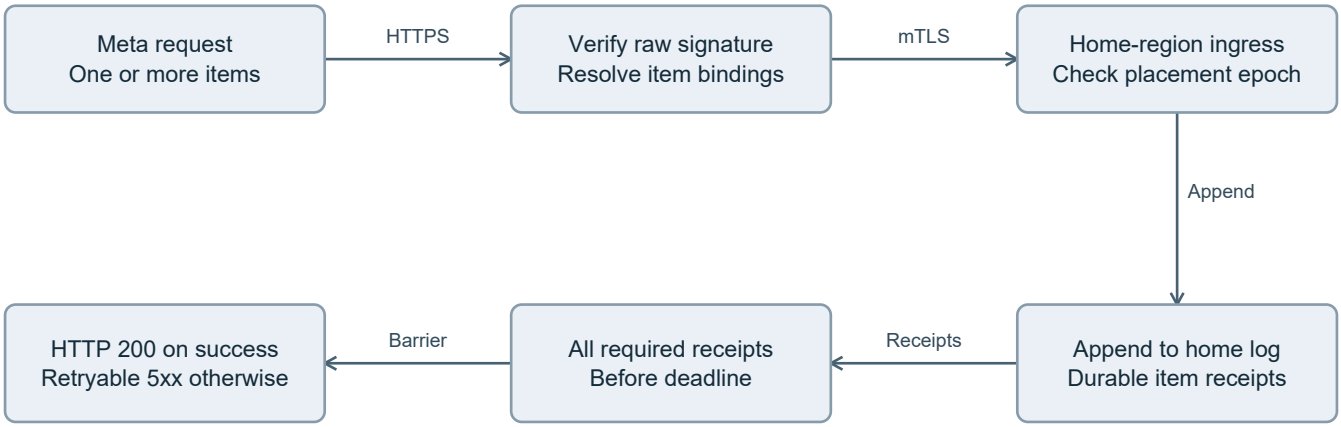


Figure 4. A forwarding ingress is not a queue. It acknowledges only after every accepted item has a durable receipt in its authorized destination region.

Condition	Response and recovery
All routed items durably accepted	Return 200. Duplicate item IDs may return an existing receipt; no duplicate business effects.
Forwarding fails or exceeds deadline	Return retryable 5xx within the provider deadline. Keep no unapproved cross-region body spool. Meta retry is bounded; alert before that horizon.
Some items committed but another failed	Return 5xx for the whole request. A repeat may redeliver committed items; destination dedupe absorbs repeats.
Append committed but receipt response lost	Return 5xx. Retry may append again or obtain an existing receipt; downstream dedupe prevents repeated effects.
Stale placement epoch	Refresh mapping once within the deadline; forward to current home. Never redirect an event into two active writers.
Authenticated but unknown asset or event	Persist to the app's authorized quarantine location, if residency is known. Otherwise fail closed and alert; do not silently drop or guess a tenant.

A durable item receipt contains app_id, item fingerprint, destination, placement_epoch and committed log position. Derive item identity from provider IDs plus event-specific stable fields, not the retry arrival time. Preserve original request hash, signature evidence and item path in the internal envelope. The router signs that envelope; home ingress authenticates it and validates the binding. A split payload does not carry the original Meta signature.

A request can contain multiple tenants. Use bounded parallel fan-out with an all-items barrier and independent destination dedupe. Adopt region-specific application/onboarding configurations when a customer prohibits even transient routing outside its region; merely avoiding disk writes is not sufficient for that residency requirement. Monitor retry exhaustion and provide an incident process for events no longer recoverable from the provider.

6 Agency organizations and delegated access

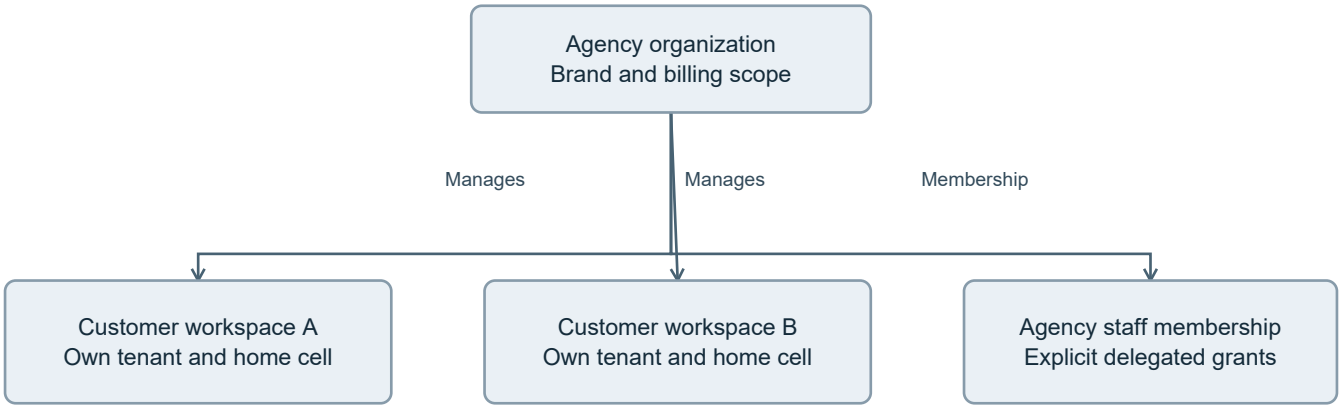


Figure 5. Hierarchy is an administrative relationship. It never grants implicit access to child transcripts, credentials or cross-customer messaging.

Use organization(id, kind, parent_id), workspace(tenant_id, organization_id, placement), membership(user_id, organization_id), and delegation_grant(principal, target_tenant, scopes, expiry, issuer). Validate acyclic hierarchy and cap its supported depth. Each child workspace remains an independent tenant with its own consent, data retention, WABA bindings, quota and encryption boundary.

An agency operator selects a child workspace before reading or acting. The server checks a current explicit grant and issues a short-lived session constrained to that tenant. Parent membership alone grants no transcript access. Background jobs carry tenant_id plus delegated actor and grant version; revocation invalidates sessions, active streams and queued privileged jobs. Customer administrators can inspect and revoke agency access.

Branding and billing

Store brand_profile with verified domain, approved logos, theme, support address and email configuration. Verify DNS ownership before binding a domain, automate TLS renewal, allowlist redirect URIs and origin headers, and retain tenant-specific host-only sessions. Never inject arbitrary tenant JavaScript or trust an unverified Host header to select a tenant. Keep identity-provider federation isolated from visual branding.

Separate usage ownership from payer identity. Each ledger record belongs to the child tenant; a billing agreement maps it to a payer and effective-dated reseller terms. Aggregate invoices through scoped financial projections. Spend caps can apply at child and parent levels, but parent finance visibility does not expose message content. Provider asset ownership is separately verified; agency tenancy must not fabricate provider portfolios or entitlement separation.

Customer transfer and offboarding

Transfer agency management by an approved, audited workflow that revokes old grants, changes billing/brand bindings, rotates delegated API credentials and rechecks provider authorizations. Preserve child data and consent history. WABA or number ownership transfers use the provider’s separate supported process. Test that a departed agency cannot query exports, push devices, search results or customer webhooks.

7 Service boundaries and ownership

Service group	Owns	Key interface
Identity and tenant control	Agency hierarchy, grants, branding, users, SSO, placement and entitlements	OIDC / SAML bridge, SCIM, signed placement snapshots
Channel account service	Portfolio/WABA/number bindings, capabilities, quality, credentials and API contract	Onboarding commands; channel.account.changed
Webhook ingress and normalization	Signature validation, durable raw envelope, provider event parsing	Provider POST; message.received / status.observed
Conversation and inbox	Typed messages, assignment, canned replies, labels, SLA instances and session window	Reply and claim commands; conversation.updated
Outbound gateway	Send intents, attempts, rate admission, provider requests, ambiguity	message.send.requested; message.dispatch.updated
Contacts and policy	Identities, imports, tags, segments, bulk jobs, consent, suppression and templates	authorizeSend; consent.changed
Campaign service	Run snapshots, schedules, recipients and ad-referral attribution	campaign.start; recipient.enqueue
Automation service	Draft graphs, immutable versions, execution history, timers	flow.publish; execution.signal
AI and knowledge	Document versions, retrieval, inference policy, evaluated prompts	Answer or handoff result; tool authorization request
Integration and media	Connector credentials, sync cursors, webhook delivery, scanned assets	connector.job; media.ready
Realtime and analytics	Authorized fan-out, push intents/devices, read models and aggregate metrics	Tenant-scoped stream; analytics queries
Billing and audit	Usage ledger, entitlements projection, administrative evidence	usage.recorded; audit.append

These are ownership boundaries; deploy them as independently scalable groups, not a service for every table. Each group owns its schema and migrations. Other groups call an API or consume a versioned event rather than writing its tables. A cell may share a physical PostgreSQL cluster initially, but it must enforce roles, connection budgets, and schema ownership. The outbound gateway remains the only service permitted to call the provider message-send endpoint.

8 Tenant onboarding and channel connection

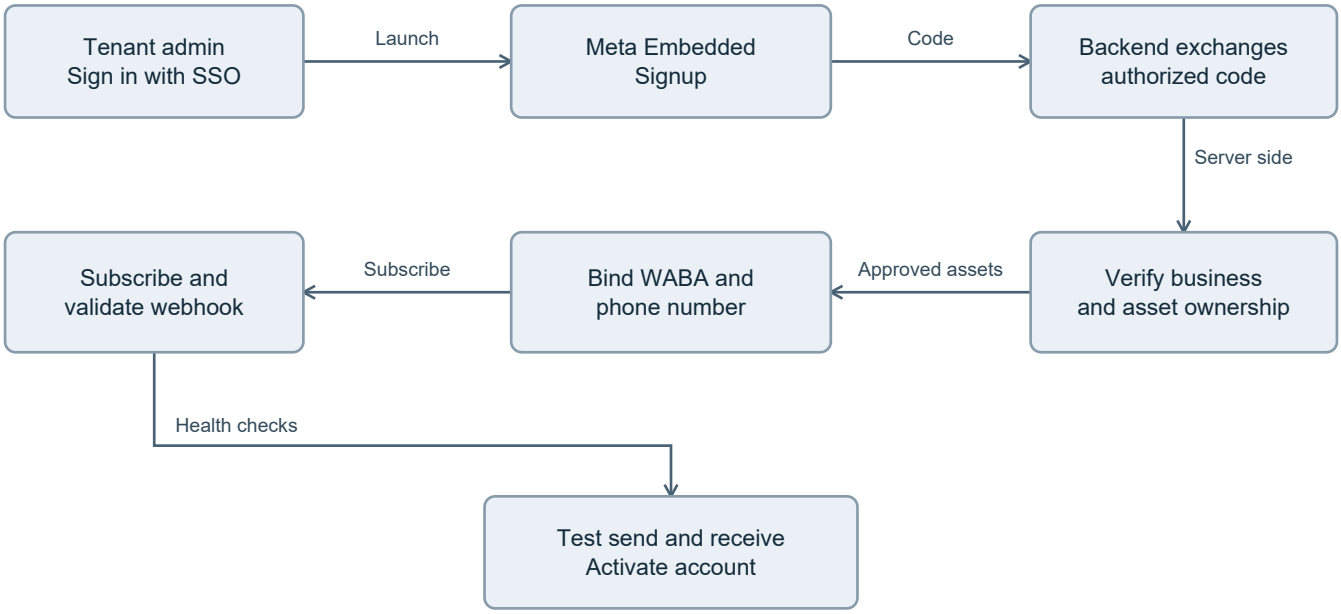


Figure 6. Activation is a resumable workflow. Every stage has a stored result, retry policy, and recoverable error state.

• Create the organization, home cell, administrators, retention policy, and entitlement record before connecting a channel. Bind signup to a server-generated nonce and authenticated tenant session; validate callback origins and expiry.

• Use Meta's supported Embedded Signup path for the approved partner model. Exchange the authorization code on the server and independently verify the granted assets. Do not trust a browser-supplied tenant or phone number mapping. [3, 4]

• Store WABA ID, phone number ID, business ID, credential reference, approved permissions, token health, and selected API version. Keep credentials encrypted in the secrets service. Permission scopes and token lifecycle depend on the integration path.

• Register or connect the number as required, subscribe the application to the appropriate account events, configure the verification endpoint, and perform real inbound and outbound checks. Activation requires both directions to work.

• Represent account state explicitly: pending_authorization, pending_verification, configuring, testing, active, degraded, revoked, disconnected. Revocation pauses sends and raises an actionable alert; it does not silently erase history.

Coexistence with the WhatsApp Business app and account migration need dedicated onboarding branches and eligibility checks. Do not assume every number supports them. Complete Meta application review, required access approvals, business verification, privacy/deletion routes, and commercial onboarding before general customer signup. [3, 4]

9 Inbound webhook pipeline

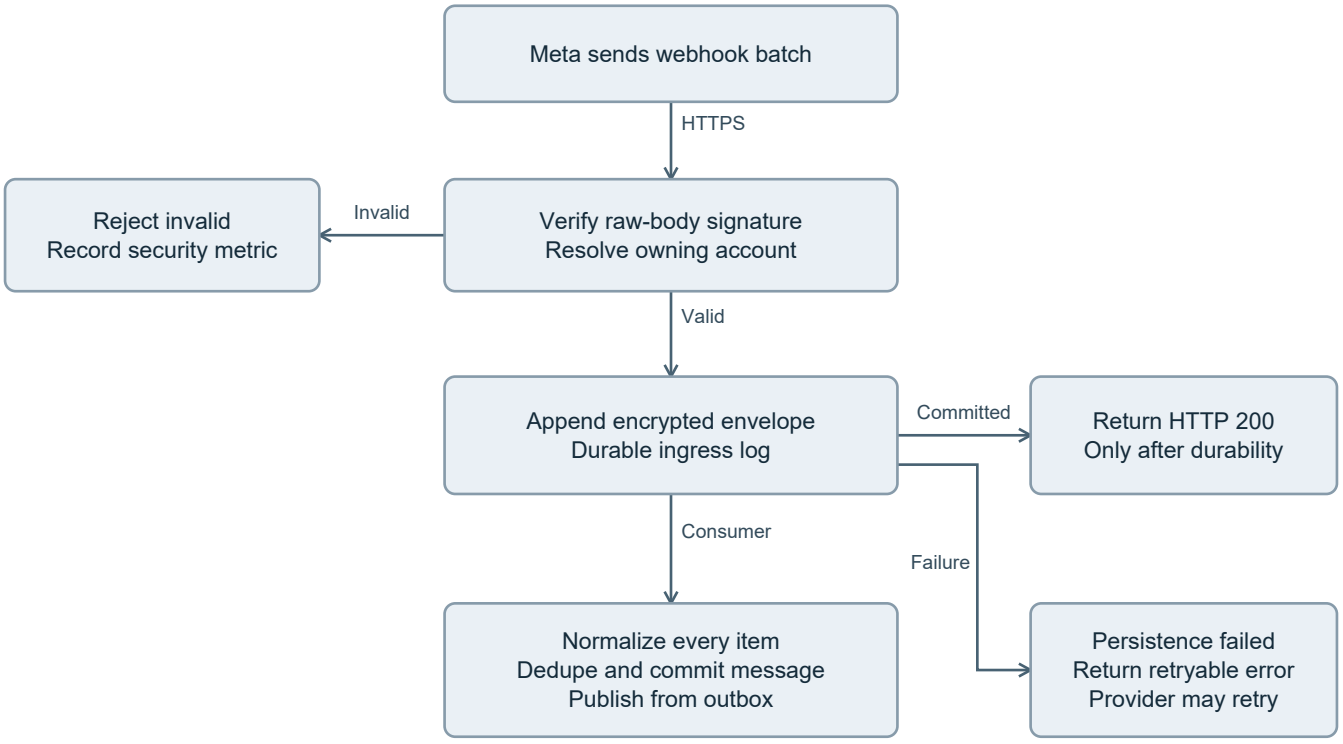


Figure 7. Fast acknowledgment means durable acceptance, not completed bot processing. The provider is never kept waiting for AI, CRM, or media downloads.

Validate HMAC-SHA256 over the original request bytes using the correct application secret and constant-time comparison. The GET subscription challenge is a separate setup check. Limit payload sizes, validate content type, and safely parse all entries and changes in a batch. Preserve authenticated unknown event types for bounded investigation instead of breaking the whole batch. [6]

Append a compact encrypted envelope to replicated Kafka with acknowledgments from the configured in-sync replica set. For a request spanning partitions or tenants, acknowledge only when every required append succeeds; partial retries are handled by deduplication. If storage is unavailable, return an appropriate retryable failure and alert. Never acknowledge first and place the event only in memory.

The key-transition pipeline on the next topic resolves provider participant identity before conversation ordering. Its owning conversation worker inserts the message, updates last_customer_message_at using the maximum valid provider timestamp, increments a conversation sequence, and writes an outbox row in one transaction. Uniqueness on tenant, channel account, and provider message ID rejects duplicate inbound delivery. Commit consumer progress only after durable work; retries may occur.

Status observations have a dedupe identity including provider message ID, status and provider timestamp or stable payload hash. This is distinct from their routing/partition key, defined in Ingress keys and conversation ordering. Archive the raw envelope by retention policy, with a pointer for diagnostics. Realtime, automation, analytics, and integrations consume independent downstream subscriptions.

10 Ingress keys and conversation ordering

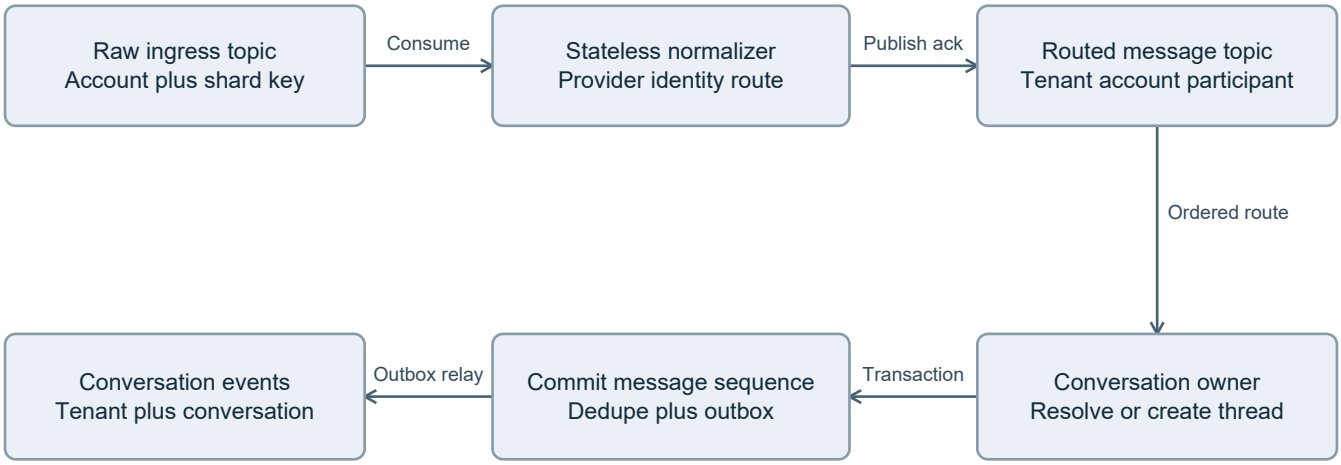


Figure 8. Raw ingress has no conversation key. Explicit repartitioning precedes transactional thread resolution; the final topic orders committed conversation events.

Stream	Partition key and guarantee
ingress.raw.v1	App + WABA/number binding + bounded hash shard from item identity. Spreads hot accounts. Arrival order across shards is not promised.
messages.routed.v1	Tenant + channel account + provider participant identity. The normalizer derives identity without requiring conversation_id.
status.routed.v1	Tenant + channel account + provider message ID. Resolve to stored send intent/conversation; buffer bounded orphan receipts until mapping exists.
management.routed.v1	Provider app + scope kind + scope ID. Account, portfolio, number and template events bypass conversation routing.
conversation.events.v1	Tenant + conversation_id. aggregate_seq is allocated by the owning service and outbox relay emits each aggregate in sequence.

The normalizer publishes with acknowledgment before committing its raw-log offset, retaining the stable event identity on retries. Conversation workers serialize each participant route and atomically resolve the thread under a unique open-thread binding, insert dedupe/message rows, advance aggregate_seq and write outbox rows. A late event belongs to the correct thread by an explicit reopening policy, not an assumed global arrival order.

Repartitioning orders processing, not real-world creation time. Preserve occurred_at and received_at, and use bounded reorder rules where business semantics demand them. Buffer a receipt that precedes the HTTP send response; never guess its conversation. Account-management consumers retain their own sequence/freshness logic. Use versioned topic migration and drain barriers when changing partition counts so one key is not consumed concurrently under two assignments.

11 Outbound message pipeline

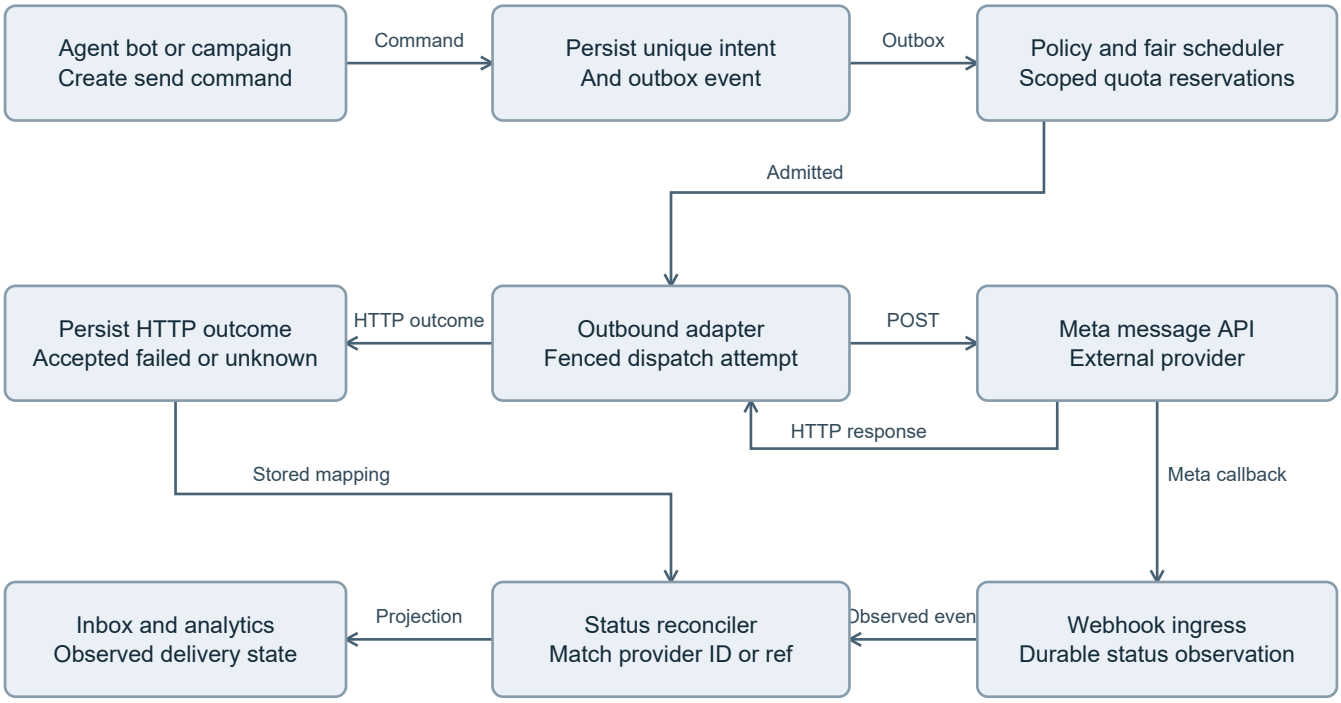


Figure 9. Meta independently originates receipt callbacks. HTTP acceptance and observed delivery are separate evidence; the following dispatch state machine defines every exception path.

Authenticate the caller, validate tenant ownership and hash the request under a tenant-scoped idempotency key. Persist the intent and outbox in one transaction before returning HTTP 202. The UI displays queued; neither 202 nor a provider message ID proves delivery. Every bot, campaign, agent and public-API send passes through this gateway.

Immediately before dispatch, evaluate consent, template/window eligibility, provider portfolio/account/number health, policy freshness, sender and recipient rate controls, ownership generation, expiry and spend. The scheduler reserves capacity at every applicable shared scope. Deferred work remains durable; policy blocks become visible terminal outcomes rather than infinite retries.

Claim a fenced dispatch attempt, persist it, then call Meta with a deadline. Save accepted provider IDs and supported opaque correlation values. The status reconciler can receive a callback before the HTTP response; it durably buffers unmatched observations and joins when the mapping arrives. The adapter never generates a provider delivery receipt. [5, 14]

On an uncertain request outcome, mark unknown and enter reconciliation. Only evidence-backed retry or an audited manual decision can leave this state. The next page defines reservation settlement and the distinction between an initial API rejection, a later delivery failure and an unresolved timeout.

12 Dispatch outcomes and reconciliation

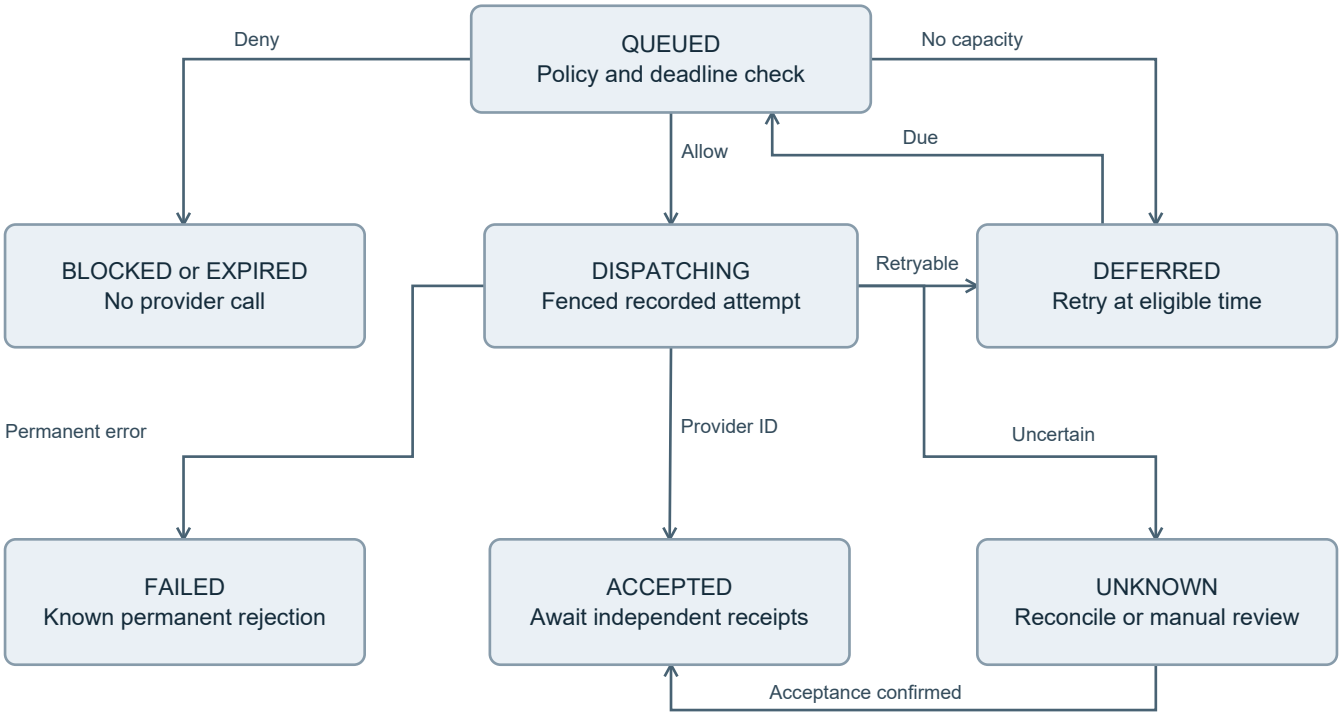


Figure 10. Deferred and documented retryable rejections return to policy evaluation when due. Unknown outcomes never return to automatic sending merely because a timer expired.

Evidence	Next action
Provider accepted request	Link provider ID; settle local submission accounting. Receipts later yield sent, delivered, read or failed observations.
Documented retryable rejection or proven pre-transmission failure	Persist retry_at with jitter and bounded attempts; return through policy and budget evaluation. Respect Retry-After.
Permanent rejection or invalid destination	Fail intent with a normalized reason. Change recipient reachability only when that specific error proves it.
Timeout/reset after possible transmission	Keep UNKNOWN; hold conservative spend/capacity reservations pending evidence. Reconcile callbacks using provider ID or supported correlation reference.
No conclusive evidence by review deadline	Remain unresolved and alert. Authorized operator may abandon or create a linked replacement with explicit duplicate risk. No claimed provider lookup API unless verified.
Accepted request later fails delivery	Record failed delivery; do not automatically create another message. A business retry is a new linked decision under current policy.

A failed attempt is not always a failed intent. Preserve attempts and observations separately; do not regress a read projection because an older sent event arrives. Audit manual resolution and reconcile usage once. A refunded local reservation does not prove the provider never charged or delivered the message; financial reconciliation retains that distinction.

13 Message capabilities and interaction context

Use a discriminated canonical message schema and a versioned capability registry for each channel, API version and connected account. The rows below are required product handling paths, not a promise that every combination is currently enabled by Meta. Validate provider support, payload limits and window/template eligibility before enabling a send control. [5, 14]

Type or interaction	Canonical data and behavior
Text and approved templates	Text body; template ID, language, revision and typed parameters. Separate template buttons from session interactive messages.
Image audio video document sticker	Media reference, MIME, checksum, scan state, caption and provider ID. Channel-specific type and size validation.
Reply buttons and list selections	Stable interaction ID and named option IDs; store displayed text separately. Incoming selection resumes the matching flow version.
Location and contacts	Typed coordinates/address or structured contact payload; explicit access and retention rules. Do not infer live-location support.
Reactions	Emoji or removal operation plus target provider message ID. Model as a message relation, not a new inbound text command.
Reply-to context	context_message_id and provider target ID. Resolve within the same tenant/account; show a placeholder when deleted or unavailable.
Catalog and product interactions	Catalog/product references and order payload; validate current price/stock in the commerce system before confirmation.
WhatsApp Flows response	Flow ID/version, scoped session token and validated form result. Route through the dedicated encrypted endpoint contract.
Ad referral and unsupported message	Preserve typed referral data when present; store safe raw reference for unknown types and display an explicit unsupported placeholder.

Interaction identity and validation

Store interaction_binding(tenant_id, outbound_message_id, flow_execution_id, flow_version_id, node_id, permitted_option_ids, expiry). Verify incoming button/list responses against this binding; a displayed label is not an authorization token. Duplicate selection events reuse event identity. Expired menus produce a safe restart or agent handoff rather than resuming a retired flow.

Store message_relation(tenant_id, source_message_id, relation_type, target_provider_id) with composite tenant/account validation. Preserve external provider participant IDs independently of phone numbers, which may be absent or change. A reply-to reference, reaction or forwarded marker must not grant access to another tenant’s content.

The web/mobile renderer escapes text, scans media, limits previews, and shows channel-specific capability errors. Define explicit fallback text for unsupported outbound interactions only when semantically safe and policy-eligible; never silently substitute a free-form message for a required template. Contract tests cover each enabled type in both directions.

14 Events ordering and reliable state changes

Canonical event envelope

```
event_id: UUID
event_type: message.received.v1
tenant_id: t_123    cell_id: eu_02
aggregate_id: conv_789    aggregate_seq: 104
channel_account_id: ca_456
provider_message_id: opaque_provider_id
occurred_at: provider timestamp    received_at: platform timestamp
trace_id: trace_abc    causation_id: event_or_command_id
schema_version: 1    data_ref: encrypted_payload_reference
```

Use a schema registry, additive changes by default, and versioned breaking events. Keep secrets and large media out of the broker. Events identify the tenant, but consumers must also validate that their credentials and storage scope permit that tenant. Sign or authenticate all producer connections; a tenant_id field is not authorization.

Consistency rules

• Partition canonical conversation events by tenant_id plus conversation_id after the explicit raw-to-participant-to-conversation key transition. Management and orphan status streams have separate keys. This keeps related events on one ordered stream, while many conversations run in parallel. Kafka ordering is within a partition, not across all topics or producers. [8]

• Allocate aggregate_seq under a row lock or optimistic version check in the owning service. Relay each aggregate’s committed outbox rows in sequence. Consumers detect gaps and reconcile; provider arrival order remains independent of business event time.

• Use a transactional outbox for database state plus event publication. The publisher can crash after publishing and before marking completion, so consumers need a durable processed-event record and idempotent effects. Broker acknowledgments alone do not make a database transaction atomic. [7]

• Commit a consumer’s processed-event marker with its business write. For remote effects, commit a stable action intent first and use the remote provider’s idempotency mechanism if available. Dedupe records must survive the full replay and retry horizon.

• Retry transient failures with exponential backoff and jitter; quarantine malformed or exhausted items with reason, original event ID, and diagnostic pointer. Replays preserve event and action identities. Never replay historical events into live sends without an explicitly authorized replay mode.

Separate ingress, conversation, outbound interactive, outbound bulk, status, integration, and analytics workloads. Use tenant quotas and weighted scheduling so a large campaign cannot consume the worker and provider capacity reserved for live replies. Poison messages block or isolate the affected conversation rather than silently violating its ordering.

15 Core data model

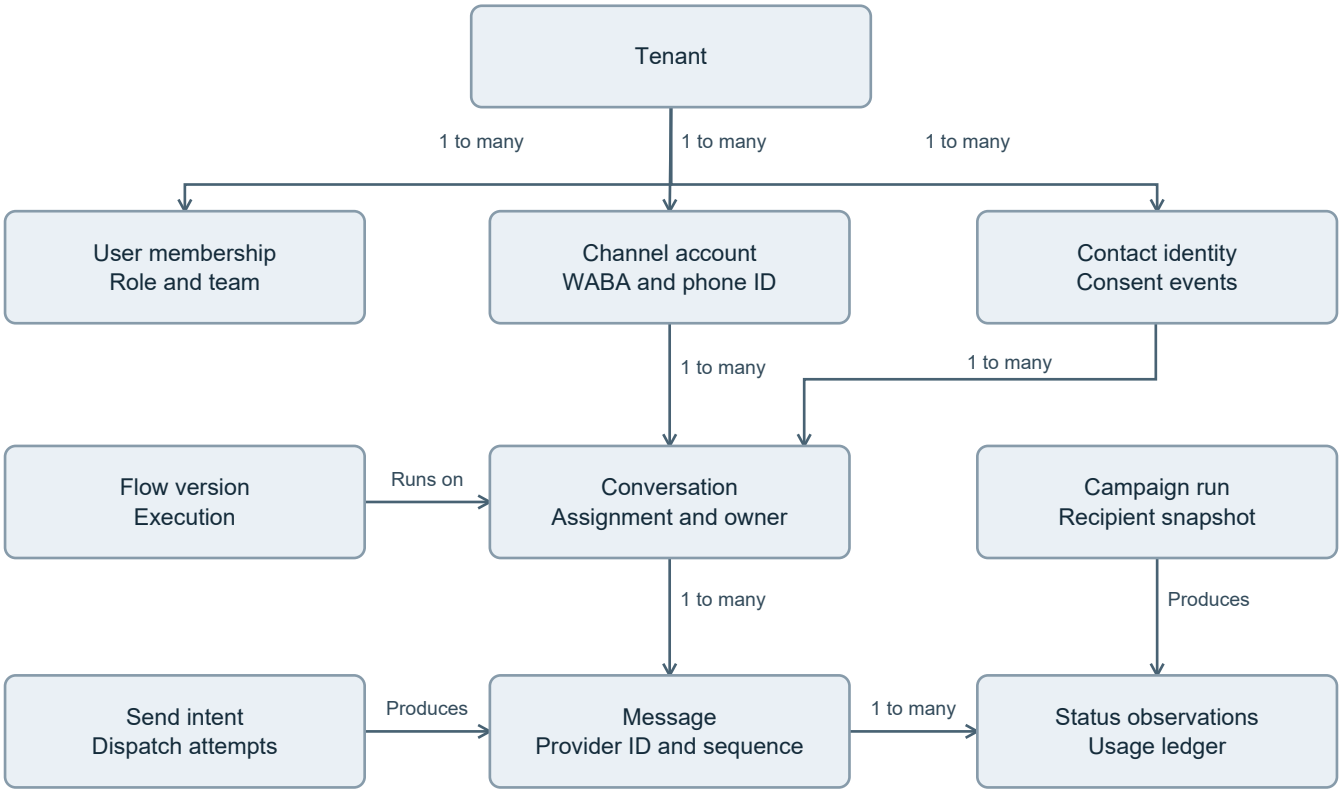


Figure 11. Conceptual relationships; every tenant-owned relation carries tenant_id. Send intents and attempts are separate from observed provider messages.

Agency organizations manage separate child tenants through explicit grants; account health, interactions, imports and SLA schemas are detailed in their owning chapters. A contact can have multiple channel identities, and a person may be represented differently across channels. Normalize phone numbers to E.164 where applicable, store provider identifiers separately, and use explicit verified merges. Do not automatically merge tenants or identities solely because display names match.

A conversation belongs to a tenant, channel account, and contact identity. Define an explicit policy for reopening versus creating a new thread, while tracking the provider's customer-service window separately from the ticket lifecycle. A closed support ticket does not mean the messaging window is closed.

Keep message bodies and attachments separate from audit evidence and analytics facts. Messages contain direction, type, sequence, timestamps, references, and retention metadata. Append provider status observations and derive the current display state without discarding late evidence. A read receipt need not be preceded by every lower status in arrival order.

Index high-volume access paths by tenant and conversation with a monotonic sequence or timestamp plus stable tie-breaker. Use cursor pagination; avoid deep OFFSET scans. Apply tenant-prefixed composite foreign keys so a valid message ID cannot accidentally reference another tenant's conversation.

16 Storage constraints and retention

Record	Required constraint or index	Retention proposal
Channel account	Unique provider + phone_number_id within the platform mapping; explicit ownership transfer	Active life plus approved deprovision window
Message	Unique tenant + account + provider_message_id; index tenant + conversation + seq	90 days hot by default; customer-configurable archive
Send intent	Unique tenant + operation + idempotency_key; request hash	At least retry and replay horizon; linked to retained messages
Dispatch attempt	Intent + attempt number; fenced claim generation	30 to 90 days operational history, then reduced evidence
Consumer inbox and outbox	Unique consumer + event_id; pending publication index	At least maximum replay horizon plus safety margin
Campaign recipient	Unique run_id + contact_identity_id; due_at index	Campaign reporting term; purge raw audience data per policy
Consent event	Append-only evidence, source, purpose, channel, observed_at	Contractual retention; current suppression retained as needed
Flow version and execution	Immutable version hash; unique execution step action key	Version retained while executions or audit need it
Media and knowledge objects	Tenant key prefix, checksum, MIME, malware state	Typically 30 to 90 days; legal hold overrides expiry
Audit and usage ledger	Append-only event ID; reconciliation keys	Term approved by security, finance, and customer contract

Physical placement

Use managed PostgreSQL per cell for transactional records, with a writer and zone-separated failover capacity. Partition large message and event tables by time and, where justified by measurements, tenant hash. Maintain a separate unpartitioned or appropriately keyed dedupe registry when a global uniqueness requirement crosses partitions. Read replicas serve historical reporting only when replica lag is acceptable.

Use Redis for ephemeral presence, cache, leases with fences, and quota counters. Use object storage for encrypted media, raw event archives, exports, and backups. Add a search index for full-text inbox queries and a separate columnar analytics store when transaction-safe reporting becomes too expensive. Derived stores are rebuildable and cannot authorize a send.

Deletion is a tracked workflow across PostgreSQL, search, object versions, knowledge chunks, analytics identifiers, and connector caches. Expire backups by policy and reapply deletion tombstones after a restore. Isolate minimal immutable audit evidence from deletable message content; avoid putting plaintext conversations in permanent logs. The durations above are proposals, not legal retention requirements.

17 Contact import and identity validation

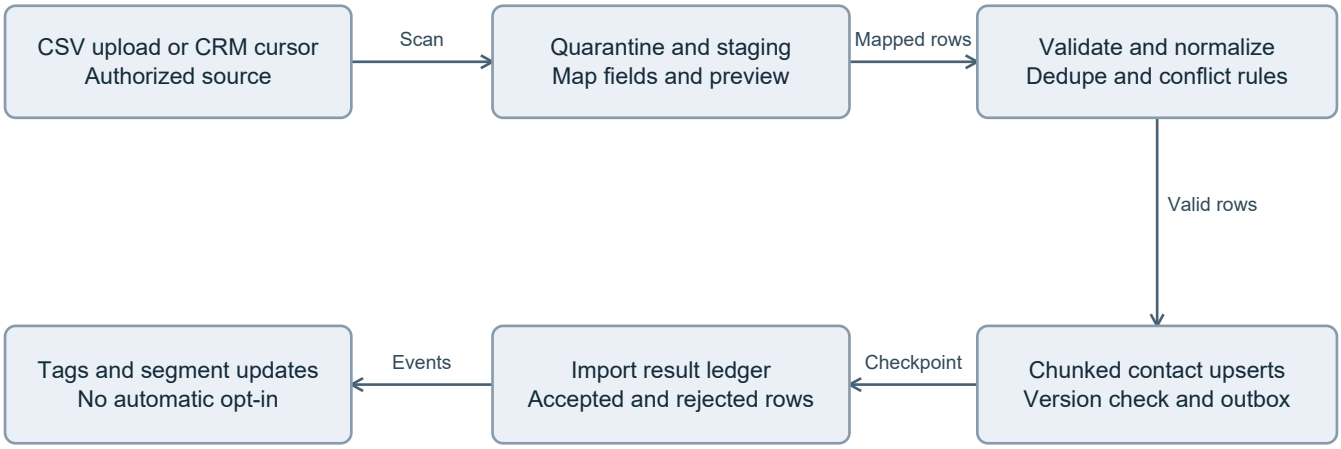


Figure 12. Importing a contact creates an identity record. It does not create marketing permission or prove WhatsApp reachability.

Create import_job with tenant, actor, source checksum/cursor, mapping version, requested conflict policy, dry-run summary and job idempotency key. Upload CSV to tenant-isolated staging, enforce bytes/rows/column limits, scan it, and parse incrementally with explicit encoding and delimiter handling. CRM imports use scoped connector credentials and saved source cursors.

The preview identifies required identity fields, custom-field types, phone-country assumptions, consent evidence and conflicting updates. Parse numbers with explicit country context into E.164 where applicable; preserve the original input and error reason. Store provider identities separately. Reject ambiguous numbers instead of guessing a country from the tenant location. Validate emails and field lengths without treating syntax as proof of ownership.

Resolve duplicate source IDs and normalized identities only within the tenant and configured identity domain. Stage merge candidates when existing identities disagree. Each row has a stable source key and processing status; workers upsert in bounded transactions with expected versions and an outbox event. Restart resumes the checkpoint; unique row/action keys prevent repeated effects. A job can finish with rejected rows and a safe downloadable error report.

Reachability and consent

Use reachability = unknown, observed_reachable, or observed_unreachable, with evidence_type, error_code and observed_at. Formatting validation is not a WhatsApp registration lookup. Do not assume a bulk registration-discovery endpoint exists, and do not send unsolicited test messages to validate imports. Learn from authorized sends and inbound activity; an ambiguous delivery failure does not prove the number is unregistered.

Consent import requires purpose, source, timestamp and policy evidence; missing evidence leaves marketing eligibility false. An import cannot clear a later opt-out. Cap import concurrency separately from messaging, encrypt staging, expire it after the agreed retention period, and escape spreadsheet-formula prefixes in exported error reports. Bulk deletions and identity merges require previewable change sets and audit evidence.

18 Tags segments and safe bulk operations

Object	Required fields and consistency
Tag and tag assignment	tenant_id, tag_id, normalized name, scope; unique tenant/object/tag assignment. Tags never act as authorization.
Custom field definition	Field ID, type, validation, visibility, index policy and schema version. Type migrations are explicit jobs.
Saved segment	Versioned predicate tree, field schema version, owner, allowed team scope, estimated count and last refresh time.
Segment membership projection	Tenant + segment + contact; source version and evaluated_at. Rebuildable and never a source of consent truth.
Bulk change job	Filter/snapshot, exact operation, preview count, actor/grant, expected versions, per-item outcome and cancellation state.
Campaign audience snapshot	Immutable run/recipient IDs and source segment version; dynamic eligibility rechecked at dispatch.

Query and update pipeline

Compile a typed segment predicate into a parameterized tenant-scoped query. Allow only approved operators and indexed fields; reject arbitrary SQL. Apply query cost, result-size and runtime budgets. High-volume segment evaluation runs in a read projection with documented lag, while consent and account policy are read from authoritative state at send time.

Support filters on tags, typed fields, channel, lifecycle stage and selected engagement facts. Define null semantics, case sensitivity, date/timezone handling and behavior for deleted field definitions. Pin the segment version when creating a campaign run and materialize recipient membership in chunks. Later contact changes do not silently rewrite the run history.

Bulk edit execution

The API creates a preview, then a durable command with the selected object snapshot and requested operation. Each chunk acquires a lease, verifies the caller’s still-valid grant and expected row version, applies changes and records per-item results. A cancellation stops new chunks; completed items remain in the result ledger. An undo job applies recorded before-images only where the record has not changed again.

Use separate job priorities for enrichment, imports, exports, retagging, suppression and deletion. A safety-critical opt-out has immediate authoritative effect even if a segment index lags. Preserve CRM field provenance and conflict precedence so a later sync does not restore deleted tags or overwrite a newer manual correction without the configured rule.

Governance and scale

Expose progress, invalid rows, conflicts, skipped items and downloadable audit results. Enforce bulk-export permissions separately from ordinary contact visibility. For agency users, resolve the child tenant before preview and execution; there is no implicit all-subaccounts bulk edit. Use bounded batches, database statement timeouts and per-tenant worker quotas so large imports and exports cannot starve live messaging.

19 Consent templates and send policy

One policy decision at every send boundary

The policy service returns allow, block, or defer with a reason code, policy version, and decision timestamp. Inputs include tenant, channel account, contact, message purpose and type, last inbound customer timestamp, template state, suppression evidence, account restrictions, schedule, and budget. Record the decision on the send intent. Recheck time-sensitive facts immediately before the network call.

Condition	Platform action
No valid marketing permission or contact opted out	Block promotional sending; preserve evidence and update suppression immediately.
Open customer service window	Permit supported response types subject to current channel rules and the user’s purpose.
Outside customer service window	Require an eligible approved template under the current provider policy; block free-form fallback.
Template paused, rejected, deleted, or wrong language	Block affected sends and mark campaign recipients with a specific reason.
Account restricted, revoked, or provider rate exceeded	Pause or defer according to error class; surface the cause to the tenant.
Quiet hours, expired campaign, frequency cap, budget limit	Defer until eligible or permanently expire; never send stale promotions.

Consent and suppression pipeline

Store permission as evidence: identity, purpose, channel, source, time, policy text/version, and revocation. A contact imported from a CSV or CRM is not automatically eligible for marketing. Match opt-out signals using configured phrases and channel controls; apply suppression transactionally before generating further automated replies or campaign work. Cancel pending promotional intents and explain any already-in-flight race in the audit record.

Template lifecycle and pricing

Store provider template ID, name, language, category, variable schema, approval status, revision, and last synchronization time. Validate sample variables at creation, synchronize status changes, and revalidate again at dispatch. A local published template is not evidence of provider approval.

Meta’s public pricing page describes delivered-message pricing and a 24-hour customer-service window. Pricing and eligibility are different rule sets: an eligible send is not necessarily free. Use effective-dated rate cards and policy configuration rather than hard-coded rates or indefinite assumptions about free message categories. [9]

20 Account health and scoped admission state

The channel account service owns a first-class provider health model. Keep provider portfolio, WABA, phone number, template and credential state separately; neither the SaaS tenant hierarchy nor a single number-level rate counter represents all provider constraints. A provider portfolio may span multiple connected numbers and must have one authoritative capacity owner.

Record	Fields consumed by admission
Provider scope binding	provider_app_id, scope_kind, scope_id, tenant grants, quota_home_cell, mapping_epoch; verified asset ownership.
Account health snapshot	registration/connection status, restrictions, alerts, credential state, observation source, observed_at, effective_at, health_version and refresh_due_at.
Number quality observation	number ID, provider quality value, raw evidence pointer and timestamp. Unknown/unavailable is distinct from healthy.
Messaging capability	scope kind and ID, limit metric/value, window semantics, usage evidence, effective period and adapter version; no fixed tier list in business code.
Template health	Template ID/language/revision, status, quality/category observations, blocking reason and last confirmed provider version.
Quota reservation	intent ID, provider scope, recipient identity, reserved units, state, expiry policy and linked provider observations; unique logical reservation.

Admission and shared capacity

The gate evaluates policy plus portfolio-level messaging capability where applicable, number throughput, account restrictions, template state, recipient controls, tenant quotas and spend. Treat unique-recipient windows, requests-per-second limits and money as different meters. A tier describes capacity, while quality and restrictions affect eligibility; they must not be collapsed into one boolean.

Route reservations to the scope’s quota_home_cell. For a strict rolling recipient allowance, use transactional recipient-window records and atomic reservations, not only a lossy Redis counter. Co-locate senders or grant bounded capacity leases whose total cannot exceed the safe allowance. If the authority is unreachable, halt bulk admission. Reconcile observations and hold uncertainty conservatively. When other apps also send under that provider scope, local usage is incomplete: refresh provider evidence, reserve headroom and treat Meta as authoritative; do not promise control over external senders.

Negative authenticated restrictions apply immediately; a stale healthy snapshot cannot override them. On stale capability/credential evidence, suspend campaigns and refresh. Permit only explicitly approved response classes under a short, documented last-known-good policy; otherwise fail closed. Store reasons visible to operators. Never automatically restore capacity from an older out-of-order quality event.

Exact current provider limit fields, scopes and event payloads must pass the adapter contract tests in the next section. The architecture deliberately supports portfolio-scoped limits and account-specific capabilities without asserting an unverified universal tier value or throughput entitlement.

21 Management webhooks and provider contracts

Subscribe and consume management events as well as messages. The following registry names are implementation candidates to verify against the selected Meta app, API version and live subscriptions. They are not a claim that every named field is available to every account today; Meta's detailed reference pages were inaccessible during this review. [17]

Provider field candidate	Normalized event and action
messages	message.received or status.observed; send to identity or provider-message routing, not the health topic.
message_template_status_update	template.status.changed; pause/reject/delete affected template admissions and alert campaign owners.
message_template_quality_update	template.quality.observed; update evidence and refresh template health.
template_category_update	template.category.changed; refresh policy and effective-dated price assumptions.
message_template_components_update	template.definition.changed; invalidate cached variable schema and reconcile version.
phone_number_quality_update	number.quality.observed; refresh the number snapshot and evaluate documented constraints.
business_capability_update	scope.capability.changed; refresh account/portfolio limit and capability configuration.
account_alerts and account_update	account.alert.observed / restriction.changed; classify severity, pause affected paths and open operator action.
account_review_update and security	account.review.changed / security.event; re-evaluate activation or credential readiness as applicable.
Unknown authenticated field	Persist quarantined schema evidence and alert; do not interpret an unfamiliar payload as healthy state.

Scope routing and freshness

Management events may identify a WABA or portfolio without phone_number_id or conversation_id. Resolve the verified app/scope binding, update the owning provider-scope record once, and fan out tenant-scoped projections only to authorized bindings. Each consumer deduplicates observations; when event time/version cannot establish freshness, refresh authoritative state instead of overwriting a newer snapshot.

Store adapter_contract(api_version, enabled_fields, required_permissions, sample_payload_hashes, provider_field_mapping, limits_source, verified_at, owner). Activation requires successful subscription readback where supported, real test payloads for required paths, and an authorized capability refresh. Pin contract versions and test additions before rollout. Periodic jittered reconciliation detects missed webhooks and drift; its rate budget is separate from message sending.

A provider restriction changing during dispatch cannot recall a call already in flight. Propagate a health generation to the last send gate, invalidate queued admission decisions and cancel unsent work. Record in-flight exceptions and reconcile their outcome. Expose stale, degraded, blocked and action-required states with evidence timestamps in the admin console.

22 Campaign and sequence pipeline

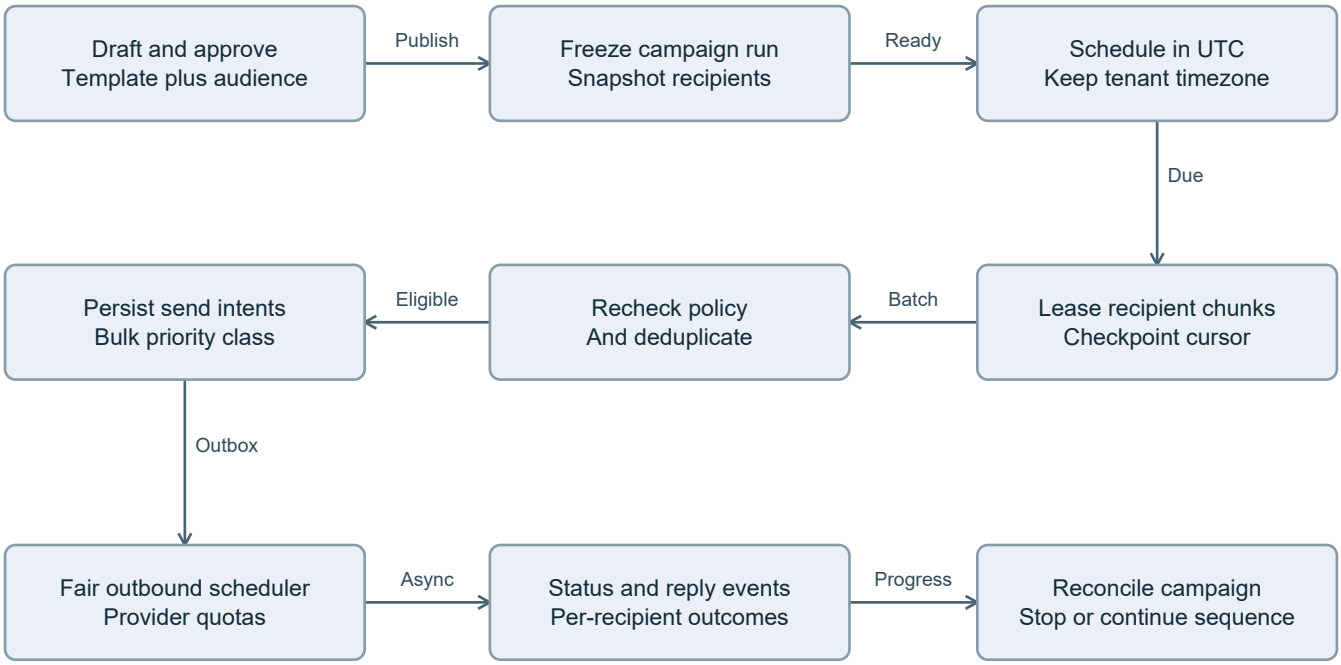


Figure 13. The audience snapshot makes the run reproducible, while a live policy check protects against later opt-outs or template changes.

Store filter definition, materialized recipient IDs, template version, variables, initiating user, approval evidence, and maximum spend. Resolve recipients in chunks so a million-contact audience does not become a single database transaction. A unique run/recipient key prevents a restarted dispatcher from enqueuing the same logical message twice.

Use weighted fair scheduling across tenants, with separate interactive and bulk queues. Apply portfolio-scoped messaging capability where applicable, sender throughput, recipient controls, tenant/agency budgets, campaign concurrency and health constraints through the authoritative scope quota service. Limits come from supported provider entitlements and current configuration, not a universal advertised messages-per-second value. Stop producers when backlog age or broker capacity exceeds limits.

Pause prevents new admissions and cancels eligible queued intents; it cannot recall requests already accepted by the provider. Keep pending, blocked, submitted, delivered, read, failed, unknown, and cancelled counts separate. Completion means all recipients have a terminal local outcome or a recorded unresolved condition, not that all recipients read the message.

For drip sequences, schedule durable timers per enrollment and stop on reply, conversion, opt-out, or agent takeover. Recheck eligibility after every wait. Store the tenant’s timezone and daylight-saving behavior for recurring schedules; convert each next occurrence to UTC instead of adding a fixed 24 hours.

23 Ad referrals attribution and conversion events

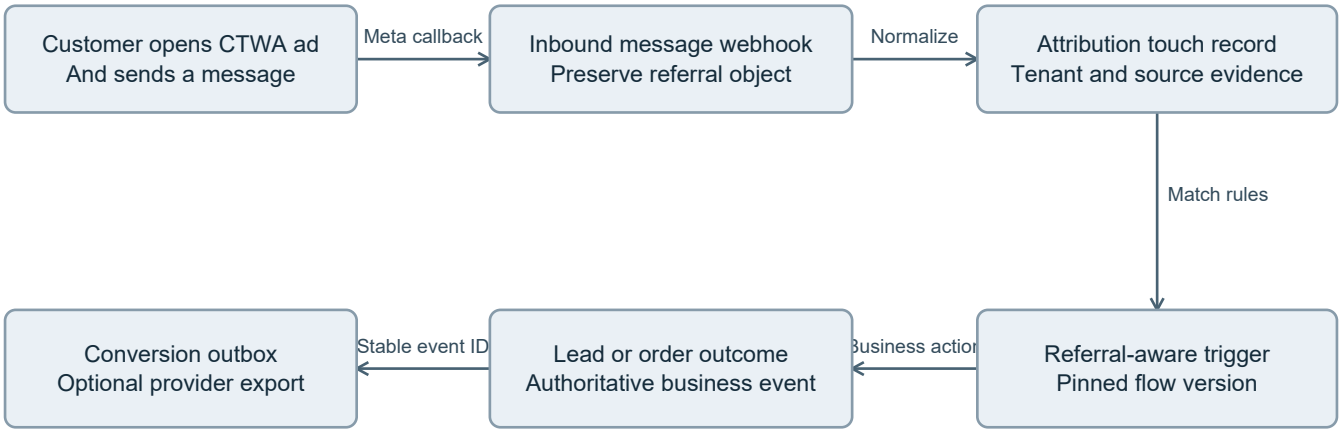


Figure 14. An ad click alone is not an inbound conversation. Capture referral evidence when it is present; missing attribution remains unknown.

Add an ad_referral trigger alongside keyword, schedule and integration triggers. The normalizer preserves a bounded raw referral reference and extracts supported source ID/type/URL and click identifier fields when present, commonly represented as referral and ctwa_clid. These field mappings are adapter-version contracts to validate; do not assume every message contains them or that they can be reconstructed later. Meta's official reference includes a CTWA-triggered message example. [14, 18]

Store attribution_touch(id, tenant_id, channel_account_id, provider_message_id, contact_identity_id, conversation_id, source_id, source_type, click_id, observed_at, raw_ref). Dedupe by the source message/touch identity. Keep multiple touches rather than overwriting a contact-level ad field. Record the configured first-touch/last-touch model, attribution window and conversion event time; report unattributed outcomes separately.

Trigger rules match approved source identifiers, campaign mappings and context, with a declared priority over generic keyword/default rules. A trigger resolution record permits one intended execution per event. Referrals do not bypass consent, template/window rules or human ownership. Treat text and URLs inside referral data as untrusted; no arbitrary fetch or tool instruction.

Lead qualification and purchase outcomes originate from authorized CRM/commerce events. Create conversion_event with stable event_id, touch reference, event time, event type and permitted value/currency. A separate optional export adapter maps it to the current Meta Conversions API contract after permissions, data-use policy and business approval are configured. Retry with event identity, record responses and reconcile accepted versus attributed; transport success does not prove an attribution result.

Raw referral storage, click IDs, conversion exports and agency reports follow tenant retention and access policy. Billing free-entry rules and marketing attribution use different evidence and must remain separate policy modules. The product offers attribution visibility even when conversion export is disabled.

24 Visual chatbot builder and publication

The graph is a versioned program

Build the editor in React and TypeScript with a graph component such as React Flow. The canvas stores a typed JSON graph, not executable JavaScript. Each node has an ID, type, input schema, output schema, validation rules, error route, and configuration. Edges refer to named output ports. A server-side compiler validates and publishes an immutable graph version with a content hash.

Node type	Execution meaning	Required controls
Trigger	Start from text, ad referral, webhook, schedule or explicit enrollment	Source match, priority, tenant scope and duplicate trigger key
Send message	Create a standard outbound intent	Template eligibility, variable typing and action idempotency
Collect input	Wait for and validate the next matching user response	Timeout, retry prompt, cancellation and sensitive-field handling
Condition and switch	Evaluate typed expressions against execution variables	No arbitrary code; bounded evaluation; explicit default branch
Wait and schedule	Resume after a durable timer or an external event	Expiry, timezone semantics, cancellation and generation check
HTTP or connector	Invoke a registered integration action	Destination policy, schema, scoped secret, timeout and retry class
AI answer	Retrieve knowledge, generate, validate, then create reply intent	Tenant filter, grounding, tool allowlist, budget and fallback
Handoff and end	Transfer ownership to an agent or finish execution	Cancel bot sends; persist reason and next owner

Compile and release

Reject missing start nodes, unreachable required steps, dangling edges, invalid variables, unsupported channel features, and loops without a bounded counter or wait. Enforce maximum graph size, transitions per event, lifetime, outbound messages, and parallel branches. Keep secrets as references to approved connectors. Simulate the graph with canned events and clock control before release.

Draft → validated → approved → published → retired is the publication lifecycle. An execution stays pinned to its starting version. A new publish changes new executions only; migration of active executions is a separately tested operation. Record who changed and approved the flow. A canary release can route a small tenant-selected share of new sessions to a version with an immediate rollback switch.

The platform’s automation graph is distinct from WhatsApp Flows, Meta’s in-chat structured forms. Add WhatsApp Flows as a channel feature with a separate definition, publication, endpoint-security, and response adapter. Do not treat a canvas graph as a provider-native form. [3]

25 Durable automation runtime

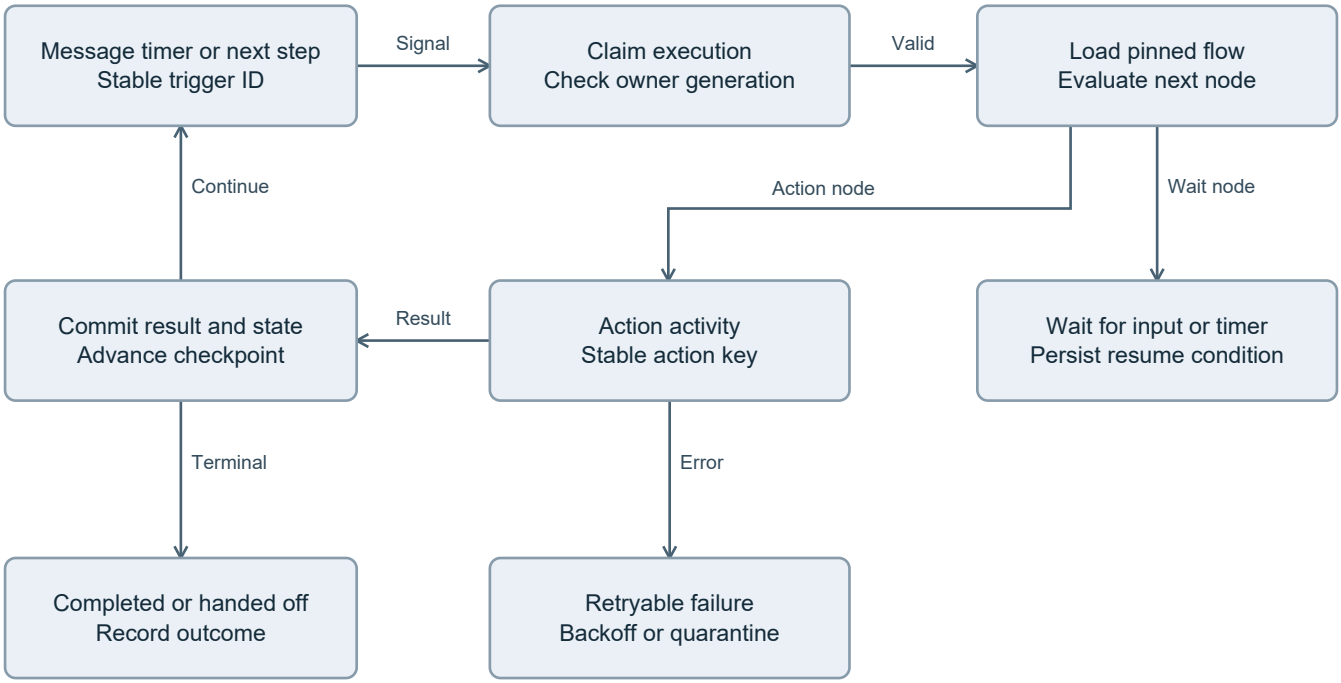


Figure 15. State survives restarts. A stored wait resumes through a new event. Activity results are recorded and retries reuse the same action identity.

Use a durable workflow engine such as Temporal for long-lived conversations, waits, and integration orchestration. Run its frontend, history, matching, and worker services with managed persistence across zones, or select a managed deployment that meets the tenant's residency requirements. Kafka distributes events; the workflow engine owns execution history and timers. They serve different purposes. [10]

Use a stable workflow ID derived from tenant and execution ID. The signaling adapter records delivery progress so a crash after signaling can retry safely; the workflow deduplicates incoming event IDs. Continue or segment long histories before their configured size limit. Encrypt sensitive payloads and store large bodies by reference.

Workflow decisions must be deterministic on replay. Network calls, wall-clock-dependent business lookups, AI inference, and random IDs belong in recorded activities or supported workflow primitives. Activities may retry, so every side effect uses an action key such as tenant/execution/node/iteration. A database uniqueness check prevents duplicate intent creation. [10]

Human takeover advances the conversation owner generation. The runtime stops or parks the flow; outbound dispatch checks the same generation immediately before sending. Already-transmitted calls cannot be recalled. Timer events carry execution generation, making a stale reminder harmless after cancellation or a new session. Unhandled input follows a declared fallback rather than starting overlapping bots.

26 Shared inbox and realtime collaboration

Read and write paths

The browser first loads an authorized conversation snapshot from the inbox API and receives its sequence cursor. It then subscribes through the realtime gateway. Each streamed event carries conversation ID and sequence; the client detects missing sequences, reconnects with its last cursor, and fetches a delta or full snapshot. Realtime fan-out may use ephemeral pub/sub because the durable database remains the recovery source.

User action	Server behavior	Concurrency rule
Open inbox	Return paginated conversations within role and team scope	Server filters every query; UI filtering is not security.
Claim conversation	Compare expected assignment version and set owner	One successful claim; stale callers receive conflict and new state.
Type a reply	Publish expiring presence after subscription authorization	Presence is advisory; it never grants ownership.
Send reply	Persist message intent with client-generated idempotency key	Check current owner/version; queued status until provider evidence.
Take over from bot	Advance ownership generation and cancel pending automation	Recheck generation at dispatch; log in-flight exceptions.
Mark read or close	Update the agent read cursor or ticket state	Do not overwrite another agent’s cursor or provider receipt.
Reconnect	Resume from last durable sequence or load snapshot	Deduplicate by event ID; bounded buffers and backpressure.

Agent routing

Route by team, language, skills, working hours, availability, and explicit capacity. Assignment service claims a conversation using optimistic concurrency and records the decision. A round-robin counter alone is insufficient when agents disconnect or reach their workload limit. Requeue abandoned assignments after a configured lease and preserve customer priority and SLA age.

A per-conversation ownership state machine is BOT_ACTIVE → HANDOFF_PENDING → AGENT_ACTIVE → RESOLVED, with explicit transitions back to automation. Treat internal notes, draft replies, and customer-visible messages as separate object types and permissions. Restrict transcript export and phone-number visibility independently of conversation access.

Realtime infrastructure

Spread gateways across zones, cap subscriptions per connection, heartbeat idle sessions, and drain connections on deploy. Do not depend on sticky sessions for correctness. Disconnect slow clients with a resume cursor instead of allowing unlimited memory growth. Revalidate authorization on subscribe and permission changes; revoke existing streams when team membership or tenant access is removed.

27 Agent productivity SLA timers and push

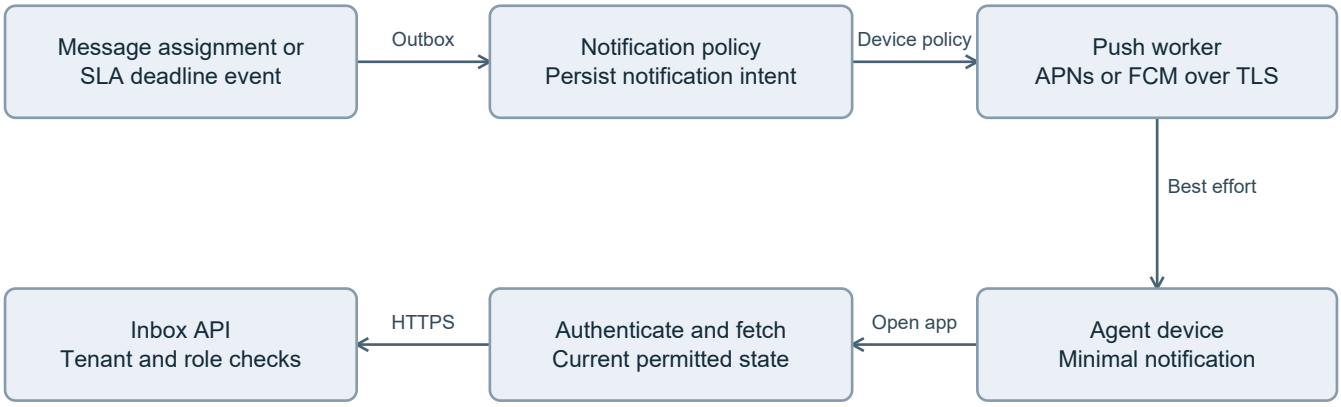


Figure 16. Push is an attention signal, not message storage or proof that an agent read a conversation. The app resynchronizes from the authorized inbox API.

Object	Required behavior
Canned reply and macro	Versioned text, language, permitted teams, variables and optional approved actions. Preview substituted content; apply normal send policy.
Conversation label	Tenant-owned label and assignment table with audit and search projection. Separate from authorization and contact tags.
SLA policy and instance	Policy version, calendar/timezone, first-response/resolution target, qualifying event, due_at, pause intervals and generation.
Escalation and timer	Durable due event, unique SLA/generation/step key, team escalation and cancellation when the qualifying condition changes.
Device registration	User, application installation, push provider, encrypted token, permission, last_seen and revocation. Never use tenant-wide public topics.
Notification intent	Tenant, recipient user, reason, object reference, dedupe key, expiry, attempts and observed provider outcome.

Calculate SLA deadlines from versioned business calendars, holidays and tenant timezones. Define whether bot responses count and whether waiting-for-customer pauses a timer. Persist deadlines and generation; a worker rechecks state before escalating. Reassignment, closure or a qualifying reply invalidates stale timers. A dashboard shows active, paused, met and breached states with the underlying event times.

Notification workers check current assignment and membership, suppress redundant alerts for active viewers, apply quiet hours where compatible with the escalation policy, and collapse replaceable updates. Use APNs for iOS and FCM for Android, with Web Push where supported. Handle invalid tokens, permission revocation, TTL expiry and retries. Providers may delay or reorder delivery; the app fetches unread state after reconnect. [15, 16]

Default push content contains no customer phone number or message body. Logout/revocation removes the device binding and prevents new pushes, while already-issued signals reveal only minimal context.
White-label native apps require separately managed app identities, credentials and device bindings; a custom web domain alone does not create a branded native app.

28 AI knowledge and response pipeline

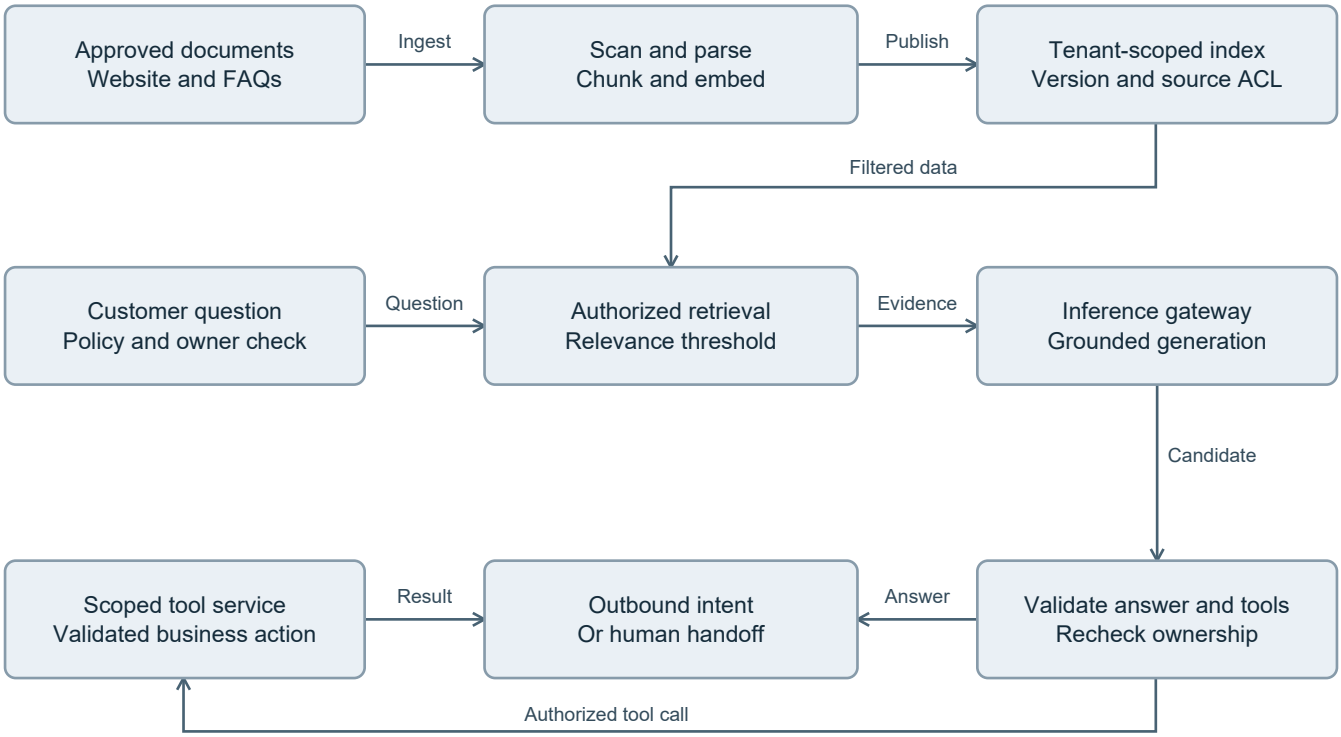


Figure 17. Documents and model output are untrusted inputs. All data access and business actions pass deterministic authorization outside the model.

Knowledge ingestion validates file types, scans malware, parses in a sandbox, strips active content, chunks text, and attaches tenant, document version, source, access tags, and checksum. Website ingestion uses a restricted fetcher with crawl limits and SSRF controls. Build a new index version and switch atomically; deleted sources invalidate their chunks and cached answers. [12]

At inference, identify the tenant and conversation from authenticated context, select only authorized sources, and retrieve bounded evidence. Pass a short relevant conversation summary and explicit business policy to the model. Require source-linked internal evidence for factual replies; use a tested relevance threshold and abstention route rather than treating the model's self-reported confidence as reliable.

The model proposes an answer or a typed tool call. A separate service validates tool name, argument schema, tenant ownership, user entitlement, and action policy. Read-only order lookup and a refund are different permissions. Sensitive actions require the business's configured confirmation or approval. External tools use scoped credentials and idempotency keys; retrieved text cannot grant new capabilities.

Define latency and token budgets, model/provider routing, data-region constraints, redaction, bounded retries, and fallback to an agent. Keep AI out of the webhook acknowledgment path. Record model version, prompt version, retrieved document IDs, tool decisions, latency, and usage with retention controls. Evaluate hallucination, tenant leakage, injection resistance, task completion, and handoff quality before auto-send.

29 Integration media and commerce pipelines

Connectors and customer webhooks

Implement a connector contract: authorize, refresh_credentials, validate_config, execute_action, pull_changes, normalize_event, and health. Each installation has tenant ownership, credential reference, scopes, object mapping, cursor, and sync direction. Use provider webhook notifications when available and scheduled cursor-based reconciliation to recover missed updates. Store source version and provenance to avoid endless CRM-to-platform-to-CRM update loops.

Inbound customer webhooks validate the connector’s signature, persist an event, and acknowledge quickly. Outbound customer webhooks include event ID, timestamp, schema version, and an HMAC signature over a documented canonical payload. Retry with backoff and a bounded horizon, then expose a delivery log and replay action. Receivers must deduplicate; your platform cannot promise exactly-once delivery to a customer endpoint.

Protect HTTP nodes, website crawlers, media fetchers, and webhook delivery against SSRF. Allow only approved schemes and destinations; reject private, loopback, link-local, metadata, and other non-public addresses for both IPv4 and IPv6. Resolve and validate at connection time, control redirects, and revalidate each redirect. Use an egress proxy to resist DNS rebinding and prevent access to private service networks. [12]

Media transfer

Inbound media metadata creates a download job. A restricted worker retrieves the provider object with a scoped token, enforces byte limits, sniffs MIME type, scans and optionally transcodes in isolation, encrypts the result in object storage, and emits media.ready. The inbox shows a pending placeholder until scanning passes. Quarantine failed assets without blocking unrelated text messages.

For outbound uploads, issue a short-lived signed upload permission for a random tenant-scoped key with a size cap. Finalization verifies ownership, checksum, file type, and scan state. The provider adapter uploads or references the approved media using the channel’s supported mechanism. Authorized downloads use short-lived links; do not log signed URLs or make conversation attachments publicly readable.

Catalogs and transactional systems

Treat CRM, order management, stock, and payment systems as the source of truth for their domains. Cache catalog displays with a version and freshness limit. Confirm price and availability against the transactional system before creating an order. Never mark an order paid from a chat message; use a verified payment-provider event and reconcile independently. Payment details should remain on the payment provider’s hosted collection surface.

WhatsApp Flows with data exchange need a separate endpoint, tenant-bound session token, replay/expiry checks, and the provider’s required encryption implementation. The underlying booking or order action still passes through the same authorized, idempotent connector service. [3]

30 Security and tenant isolation

Identity and access

Use an established identity provider for OIDC login and enterprise SAML federation; support SCIM provisioning where required. Require MFA for administrators and privileged operators. Maintain tenant memberships separately from global identity, and issue short-lived sessions or tokens with audience and scope. Recheck server-side roles for every operation and audit role changes, exports, credential changes, and flow publication.

Define owner, administrator, supervisor, agent, analyst, developer, and billing roles, with team and conversation-level restrictions. Service-to-service calls authenticate workload identity and propagate a validated tenant context. The API gateway strips externally supplied internal identity headers before adding trusted ones. Customer API credentials are scoped, hashed at rest where possible, rotatable, and never equivalent to operator credentials.

Database and storage controls

Enforce tenant_id in queries and composite keys, backed by PostgreSQL row-level security on tenant tables. Use non-owner application roles without BYPASSRLS; FORCE ROW LEVEL SECURITY where appropriate. Set tenant context locally inside a transaction and reset pooled connections. Admin and migration roles stay separate. Test cross-tenant reads, writes, joins, exports, background jobs, and error messages. RLS is defense in depth, not a substitute for application authorization. [13]

Scope object storage paths, signed URLs, search filters, vector retrieval, caches, broker ACLs, and realtime subscriptions. A missing tenant filter in a search or AI index can bypass an otherwise secure database. Use dedicated credentials and encryption keys for higher-isolation tenants. Encrypt disks, objects, backups, and sensitive workflow payloads; keep secrets in a managed vault with rotation and audited access.

Privileged access and application defenses

Give support staff metadata-only access by default. Require a time-limited, auditable elevation for customer content, with tenant approval where the contract requires it. Keep a separate admin domain and private operator path. Use just-in-time access, session recording where supported, and emergency access reviewed after use.

Escape rendered messages and notes, sanitize HTML, enforce a strict content-security policy, protect state-changing cookie requests against CSRF, validate uploads, and restrict URL fetchers. Rate-limit login, exports, API creation, media, and resource-intensive queries. Keep cloud metadata unreachable from untrusted execution. Isolate connector and document-processing workloads from core credentials.

Run dependency and image scanning, secret detection, SBOM generation, signed-image verification, and periodic penetration tests. Certifications such as SOC 2 or ISO 27001 are organizational programs with evidence requirements; this architecture does not confer certification. Map controls to actual customer contracts and applicable law before launch.

31 Public API and event contracts

Send a message

```
POST /v1/channel-accounts/ca_456/messages
Authorization: Bearer <scoped-platform-token>
Idempotency-Key: order-987-dispatched-v1

{
  "contact_id": "ct_123",
  "type": "template",
  "template": {"id": "tpl_22", "language": "en",
    "parameters": {"order_number": "987"}},
  "client_reference": "order_987",
  "expires_at": "2026-09-30T12:00:00Z"
}

202 Accepted
{"message_intent_id":"mi_001","state":"queued",
 "status_url":"/v1/message-intents/mi_001"}
```

Infer tenant from authenticated credentials and verify that ca_456 and ct_123 belong to it. The request contains an internal template ID resolved to a provider template by the channel adapter. This is a proposed platform contract, not a literal Meta request body. A successful response confirms durable intent acceptance, not provider delivery.

Endpoint	Behavior
GET /v1/conversations?cursor=...	Stable cursor pagination with team authorization; returns next cursor and snapshot sequence.
POST /v1/conversations/{id}/claim	Requires expected_version; returns 409 when ownership changed.
POST /v1/flows/{id}/publish	Validates graph, checks approval, creates immutable flow_version_id.
POST /v1/campaign-runs	Validates audience, template, schedule and budget; returns asynchronous run ID.
POST /v1/webhook-subscriptions	Creates a verified, signed-event destination with SSRF controls.
GET /v1/message-intents/{id}	Returns local state, provider ID if known, observed receipts and safe error details.

Use a consistent error shape: code, human-readable message, retryable flag, request_id, and field errors. Return 401 for invalid authentication, 403 for forbidden actions, 409 for conflicts, 422 for invalid payloads, and 429 with retry information for quota limits. Avoid leaking whether another tenant’s resource exists. Document idempotency retention, pagination, webhook retries, versioning, and deprecation windows.

Enforce schema and size limits at the edge and application layers. Version OpenAPI and event schemas in source control, run consumer contract tests, and provide a sandbox with synthetic contacts and provider stubs. Pin the Meta Graph API version per adapter deployment and test upgrades before changing live tenant traffic.

32 Enterprise command and storage contracts

Proposed platform endpoint	Contract
POST /v1/import-jobs	Tenant-scoped source reference, mapping version, conflict policy and idempotency key; returns job ID. Dry-run endpoint returns row errors and prospective changes.
POST /v1/bulk-jobs	Immutable selection snapshot, operation, preview token and expected schema version; returns per-item progress. Cancellation affects unstarted work.
POST /v1/segments	Typed predicate and version; query cost validation. Campaign run refers to pinned segment version and materialized recipients.
POST /v1/delegation-grants	Child workspace authorization required; explicit principal, scopes and expiry. Revocation invalidates cached grants and queued privileged work.
GET /v1/channel-accounts/{id}/health	Authorized snapshot with scope IDs, restrictions, quality evidence, freshness and policy generation. Never return provider secrets.
POST /v1/device-registrations	Authenticated app installation and provider token; bind to user/app. Rebinding or logout revokes previous associations.
POST /v1/sla-policies	Versioned targets, calendar, pause rules and qualifying response classes. Active SLA instances retain their policy version.
GET /v1/attribution-touches	Tenant-scoped paginated touches with model/window metadata; sensitive click identifiers require a separate export permission.

Storage ownership and constraints

Contacts owns import_job, import_row_result, tag, contact_tag, custom_field_definition, segment_definition and bulk_job. Conversation owns interaction_binding, message_relation, canned_reply, conversation_label and sla_instance. Notifications owns device_registration and notification_intent. Identity/control owns organization, workspace, delegation_grant and brand_profile. Channel accounts owns provider_scope_binding, capability_snapshot, health_observation and adapter_contract. Attribution owns attribution_touch and conversion_event.

Every child-owned table includes tenant_id; composite keys and foreign keys enforce tenant scope. Uniqueness examples are (tenant_id, import_job_id, source_row_key), (tenant_id, execution_id, interaction_id), (tenant_id, sla_instance_id, generation, escalation_step), and (tenant_id, conversion_event_id). Device tokens are restricted global identity infrastructure: resolve current user-to-tenant permission before every tenant notification.

Cross-tenant provider-scope and agency records live behind dedicated services with explicit binding grants; ordinary tenant database roles cannot enumerate them. Tenant projections receive only their allowed slice. Hash opaque API keys, encrypt provider/device credentials, and audit scope changes. All command endpoints use consistent idempotency, conflict, cancellation and authorization semantics from the core API chapter.

33 Analytics billing and audit pipelines

Analytics

Publish canonical events from each owning service to a regional analytics pipeline. Deduplicate by event ID, apply event-time windows with a defined lateness allowance, and write dimensioned facts to a columnar store. Separate operational inbox reads from large analytic scans. Tenant-facing dashboards must disclose freshness and distinguish provider-accepted, delivered, read, replied, and converted events.

Define metrics before implementation: delivery rate uses delivered divided by provider-accepted sends with a declared observation window; reply rate counts unique replying recipients; first-response time measures first qualifying business response after the customer’s initiating event. Exclude internal notes and document whether bot responses count. Missing read receipts are unknown, not proof of unread messages.

Metering and financial reconciliation

Create an append-only usage ledger for platform seats, accepted billable platform actions, AI tokens, storage, and any provider charges passed through. Each record contains child tenant, effective payer agreement, source event, meter, quantity, unit, rate-card version, timestamp and reconciliation status. Agency rollups consume scoped financial projections, never transcript joins. Uniqueness prevents duplicate callbacks or event replay from duplicating charges.

Prefer direct Meta billing to the tenant when the commercial model permits it. If reselling provider spend, maintain reservations, final settlement, credit notes, taxes, currency treatment, and invoice reconciliation as a separate financial domain. Never charge every send attempt as if it were delivered. Preserve estimated, observed, and invoiced cost separately and reconcile to provider evidence. [9]

The send gateway reads a local entitlement and budget projection so a billing dashboard outage does not stop live support. Expiring cache and conservative limits prevent uncontrolled spend. Campaign launch reserves a configured maximum budget; recipient settlement releases unused reservations. A subscription webhook is deduplicated and verified before entitlement changes.

Audit evidence

Record actor identity, tenant, action, target, timestamp, request ID, source IP or device context when justified, previous and new configuration hashes, and approval references. Store security-critical evidence in an immutable log account with restricted writers and retention controls. Keep sensitive message bodies out of audit entries whenever IDs and hashes are enough.

Analytics rebuilds must not trigger sends or rebill usage. Use separate consumer identities, read-only processing modes, and destination topics. Reconciliation jobs compare source message counts, intent outcomes, provider observations, ledger totals, and campaign recipient totals; discrepancies create a review task rather than an automatic customer charge.

34 Availability targets and observability

The following are proposed internal targets under the capacity envelope in Capacity model and cost drivers. Measure them before committing contractual SLAs. Provider outages, device connectivity, template review, and AI latency must be visible separately from platform availability.

Indicator	Initial target	Measurement boundary
Webhook durable acceptance	99.95% availability; p99 below 500 ms	Valid webhook arriving at platform edge to durable log acknowledgment; provider delivery delay excluded.
Core API	99.95% availability; p95 reads below 300 ms	Authenticated valid requests at platform edge; complex exports measured separately.
Message visible in inbox	p95 below 2 seconds	Durable inbound acceptance to authorized browser event in a connected session.
Interactive send admission	p95 below 2 seconds	Persisted eligible intent to start of provider request; exhausted provider quota reported separately.
Simple bot reply intent	p95 below 3 seconds	Inbound durable acceptance to persisted reply intent; external AI/CRM measured separately.
AI answer	p95 below 12 seconds; bounded timeout	Eligible AI request to answer or handoff; selected model and evidence size recorded.
Same-region zone failure	No loss of acknowledged committed records within configured durability	Verified database/broker failover test; maintain quorum and spare capacity.
Regional recovery	RPO at most 5 minutes; RTO at most 60 minutes	Target for approved cross-region replicas; replication lag and promotion drill determine achievable result.

A 99.95% monthly availability objective allows about 21.6 minutes of unavailability in a 30-day month. Use error-budget burn alerts over short and long windows. Separate platform failure from tenant misconfiguration in reporting without hiding real customer impact.

Signals and alerts

Propagate trace_id, tenant_id, conversation_id, intent_id, and provider message ID through logs and traces. Redact tokens, message bodies, and signed URLs. Use bounded metric labels rather than one time series per message or customer. Sample traces while retaining failure exemplars with access controls.

Alert on ingress acknowledgment errors, oldest unpublished outbox age, consumer lag, oldest eligible interactive intent, ambiguous sends, provider 429/5xx rates, token revocation, DLQ growth, database saturation, replication lag, and cost anomalies. Give every alert an owner, customer-impact description, dashboard, and tested runbook. Run synthetic inbound/outbound checks using controlled accounts.

35 Capacity model and cost drivers

Planning example: 10,000 tenants, 20,000 connected agents, and 20 million message records per day across 20 cells. Assume 60% outbound messages and a 10-times daily-average burst. This is an engineering sizing scenario, not an expected launch volume or a measured benchmark.

Calculation	Result and implication
Message average = 20,000,000 / 86,400	231.5 messages/s globally; approximately 11.6/s per cell at even distribution.
Message burst = average × 10	2,315 messages/s globally; 116/s per cell before hot-tenant skew.
Events = 20M messages + 3 × 12M outbound status events	56M events/day; 648/s average; 6,481/s at 10-times burst. Verify actual status multiplicity.
Event bytes = 56M × 2 KB	112 GB/day uncompressed. Seven-day log ≈ 784 GB logical; three replicas ≈ 2.35 TB before overhead.
Message rows = 20M × 1.5 KB	30 GB/day logical; 90 days ≈ 2.7 TB before indexes, replicas, WAL and backups.
Media = 5% of messages × 250 KB	250 GB/day; 30 days ≈ 7.5 TB before replication and transforms. Media dominates bytes.
Realtime heartbeat = 20,000 / 30 seconds	667 heartbeat cycles/s globally, plus inbound acknowledgments and message fan-out.
AI = 5% of 8M inbound × 1,500 input + 250 output tokens	400,000 calls/day; 600M input and 100M output tokens/day. Budget and cache by tenant.

With a burst of 324 events/s per cell, a 30-minute processing outage creates about 583,200 events of backlog. If the recovered cell sustains 800 events/s while arrivals remain at 324/s, net drain is 476/s and recovery takes about 20.4 minutes. If arrivals exceed capacity, adding time will not clear the queue.

A starting load-test configuration might use 24 ingress partitions per cell, replication factor 3, minimum in-sync replicas 2, and producers requiring all configured acknowledgments. Partition count must be validated against hot keys and consumer cost. Size databases, workers, network throughput, and connection pools for one-zone loss and skew; uniform tenant distribution is only a simplifying assumption. [8]

Monthly cost = compute + transactional stores + broker/workflow services + media storage and egress + observability + backups + AI + provider messages + support operations. Provider cost is delivered eligible messages by market/category/rate period; AI cost is model-specific token usage. Obtain dated regional quotes and run a load test before assigning a dollar budget. Dedicated cells and cross-region recovery create a material fixed baseline.

36 Enterprise capacity and isolation budgets

The preceding capacity example sizes message records and provider status observations. It is not the full broker, database or workflow workload. Version 2 adds explicit planning factors for routing stages, management events, imports, attribution and agent notifications. Measure each service's amplification before purchasing capacity or promising a customer limit.

Workload	Sizing and isolation requirement
Broker stages	Count raw, routed, canonical, outbox-derived and retry records separately. At an illustrative 3 durable copies per original event, 56M events/day becomes 168M writes/day before retries and fan-out.
Broker storage	At the same illustrative 2 KB per copy and 7-day retention, 3 stages × 112 GB/day × 7 × 3 replicas ≈ 7.06 TB. Real topics have different payload sizes, compression and retention.
Workflow execution	Measure active conversations, timers per execution, history events per turn, workflow/activity tasks and persistence IOPS. Provider-message rate alone cannot size Temporal.
Imports and segments	Budget rows/s and database write amplification by tenant. Separate worker pools, pools/roles, job queues and I/O budgets from interactive messaging.
Management and quota scope	Model hot portfolios shared by many numbers/cells. Test one authoritative reservation owner or safe bounded leases; avoid a cross-cell double-spend.
Push and inbox fan-out	Message events × authorized recipients × active devices × selected channels, with suppression/collapse factor. Track real connections and reconnect storms.
Search analytics and AI	Budget indexing lag, storage copies, vector dimension/version growth, inference tokens and backfills. None may starve durable ingress or dispatch.
Regional failover	Recovery capacity serves promoted load plus replication catch-up and retries. Pause bulk/import jobs first; reserve interactive and management capacity.

Retain the original 20-cell example as a scale envelope, not a mandatory launch count. Commission enough cells for contracted customers, hot-tenant isolation, failure headroom and approved recovery pairs before launch. Adding cells is automated fleet expansion; tenant boundary, service ownership, broker and workflow design already exist in the first production cells.

Load-test at least one hot tenant, one hot provider portfolio, one failing connector and a campaign/import burst while losing a zone. Define admission ceilings from the first exhausted resource, not only CPU. Use per-cell lag/latency signals to throttle bulk producers and trigger placement changes. Cost models include every durable copy, cross-zone/region transfer, NAT traffic, telemetry and recovery reserve.

37 Failure handling and disaster recovery

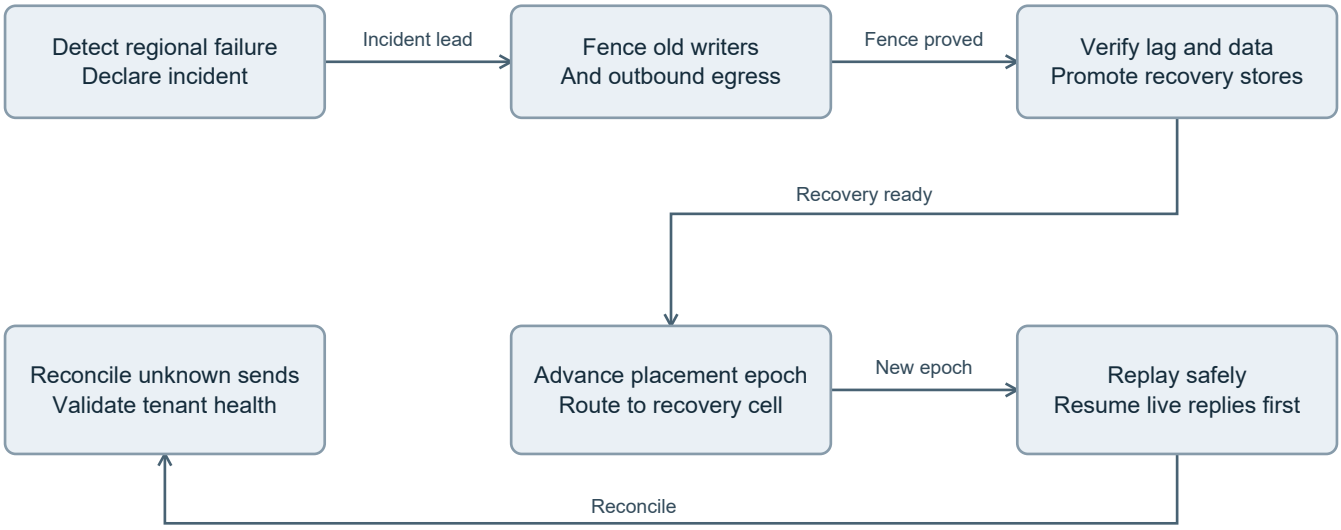


Figure 18. Promotion requires verified fencing. A region partition must not produce two active senders for one tenant.

Failure	Required behavior
Database or broker unavailable	Do not acknowledge unpersisted work. Keep committed backlog; fail readiness and alert.
Worker crashes mid-transaction	Transaction rolls back or commits atomically; redelivery is deduplicated.
Provider times out after transmission	Mark outcome unknown; reconcile evidence; do not automatically duplicate the send.
AI or CRM outage	Circuit breaker, bounded retries, explicit fallback or handoff; webhook acceptance continues.
Zone loss	Healthy replicas accept work; redistribute within reserved headroom; protect quorum.
Regional loss or network partition	Freeze bulk work, fence old writers, restore/promote stores, advance epoch, resume and reconcile.

Replicate database backups/WAL, event archives, object data, configuration, secrets references, and workflow history to an approved recovery region. Define replication and recovery independently for each store. Durable workflow state needs a supported replication or restore mechanism; rebuilding timers from an inbox snapshot is not equivalent. Redis presence can be discarded and rebuilt.

A global lease or fencing authority must prevent stale-region sends after promotion. Old dispatchers fail closed when their short lease expires; network controls or revoked regional credentials provide a second fence. If the old region cannot be proven fenced, postpone outbound promotion even if this exceeds RTO. Asynchronous replication can lose up to the measured lag, including recent dedupe state; quarantine affected send windows for reconciliation.

Run quarterly recovery drills and regular backup restores. Reapply deletion tombstones, verify record counts and checksums, compare provider observations, and test access controls before reopening all traffic. Failback is a controlled migration, never an automatic DNS flip while both regions can write.

38 Delivery pipeline and deployment

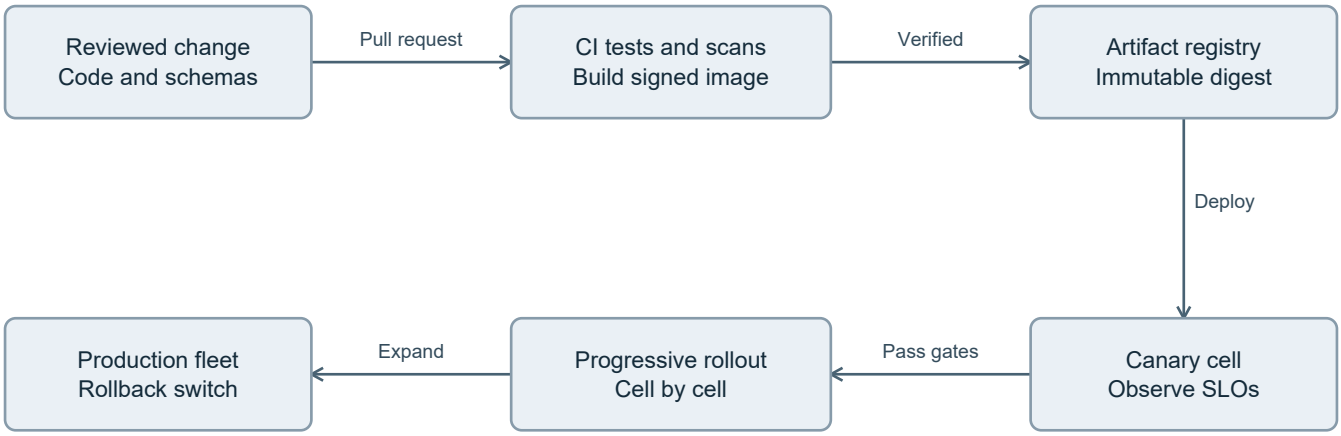


Figure 19. Infrastructure and application changes pass through the same reviewed release process; cells limit the blast radius.

Reference stack

Use React and TypeScript for the web console, Go for ingress/dispatch and high-throughput workers, and a consistent typed service framework for business APIs. Avoid adding languages without an operational reason. Use managed Kubernetes for the enterprise deployment, managed PostgreSQL and Redis, managed Kafka, object storage, a durable workflow engine, and OpenTelemetry collectors. Kubernetes is the selected reference runtime for the enterprise fleet; separate worker pools and budgets protect messaging from imports, analytics and notifications.

On AWS, the concrete mapping is Route 53, CloudFront/WAF, regional load balancers, EKS, RDS or Aurora PostgreSQL, ElastiCache, MSK, S3, KMS, Secrets Manager, and a controlled egress path. Select exact editions and regions through measured requirements; these are service choices, not a price or feature guarantee. Keep account policies, network routes, IAM, schemas, and alarms in infrastructure as code.

Safe change management

CI runs unit tests, schema compatibility, API contracts, tenant isolation tests, migration checks, dependency/license/secret scans, and signed-image production. Staging uses a provider sandbox or controlled test number, realistic sanitized loads, and separate credentials. Deployment promotes an immutable image digest; no operator rebuilds production from a laptop.

Use expand-and-contract migrations: add compatible schema, deploy readers/writers that handle both forms, backfill in bounded batches, and remove old fields only after all consumers have migrated. Back up before destructive migrations and test rollback assumptions. A binary rollback cannot undo an incompatible database deletion.

Canary on an internal tenant or low-risk cell, compare queue age, errors, latency, and message correctness, then expand. Version workflow workers and keep existing executions compatible with their recorded history. Drain long-lived connections and workers gracefully. Separate feature flags for AI auto-send, new flow versions, provider API upgrades, and campaign admission.

39 Verification and launch acceptance

Test scenario	Pass condition
Webhook duplicates and mixed batches	One canonical message per provider ID; all valid items persisted; repeat delivery does not repeat bot actions.
Invalid signature or forged tenant context	No domain state change; event rejected and safe security evidence recorded.
Crash after DB commit before publish	Outbox resumes and downstream processes exactly one logical effect after dedupe.
Crash after provider accepts but before local result	Intent becomes unknown or reconciled; no blind second send.
Status events arrive out of order	Read/delivery projection remains consistent; raw observations are retained.
Agent takes over while AI is running	Bot generation is invalidated; unsent intent is blocked by ownership fence.
Opt-out after audience snapshot	All not-yet-dispatched promotional intents are blocked or cancelled.
One hot tenant sends a huge campaign	Other tenants meet live-reply SLO; quotas and backlog controls activate.
Tenant ID swapped in API, job, search or AI query	No cross-tenant data, signed URL, action, or existence disclosure.
Workflow publish during active session	Existing session stays on old version; new session uses intended release.
Worker restart during timer and HTTP action	Wait resumes once logically; side effect uses stable action ID.
Zone loss and regional promotion	Measured RPO/RTO meet target; no two active senders; unknown window reconciled.
Deletion then backup restore	Tombstones reapplied across all stores; deleted content is not exposed.
Provider API version upgrade	Contract suite and real test send/receive pass for every supported message type.

Run sustained capacity, burst, soak, queue-recovery, and failure-injection tests. Model hot conversations, skewed tenants, large attachments, slow clients, token expiry, invalid templates, expired messaging windows, and provider throttling. Include campaigns concurrent with live support rather than testing each in isolation.

Launch evidence must include a restore report, load-test report, tenant-isolation review, security findings with remediation, provider onboarding approval, alert-to-runbook drill, and signed operational ownership. Measure AI against a curated business question set and adversarial retrieval content; approve auto-send separately from assisted drafting.

40 Review regression tests and evidence

Acceptance case	Required evidence
Network topology matches deployment	Infrastructure graph proves brokers in data boundary, application normalizers/relays in app boundary, and only approved external egress. Exercise publish and consume paths.
Independent receipt races HTTP response	Inject receipt before adapter result; observe buffered matching and one correct projection. No fake receipt from the adapter.
Every dispatch branch terminates or waits intentionally	Exercise blocked, expired, deferred, retryable, failed, accepted and unknown states; prove unknown never blindly resends.
Ingress has no conversation ID	Raw item processes through participant re-key, atomic thread resolution and ordered conversation outbox exactly once logically.
Partial cross-region forwarding failure	One destination commits and another fails; edge returns 5xx. Replay obtains existing receipt without repeated message or bot action.
Account/portfolio event lacks phone number	Route by verified scope binding; update only entitled tenants and invalidate relevant admissions.
Shared provider allowance across cells	Concurrent reservations cannot exceed a strict configured allowance; unavailable quota authority halts bulk admission.
Agency grant revoked during job	Queued export/bulk job and open streams stop accessing child data; sibling tenant never becomes visible.
Import restart and conflicting opt-out	Resume without duplicate contacts; rejected rows reported; newer suppression preserved over import evidence.
Ad referral missing duplicated or repeated later	Unknown remains unknown; duplicate touch suppressed; later distinct touch retained; only one intended trigger fires.
Buttons lists reactions and reply context	Round-trip each enabled type; invalid option/stale flow rejected; cross-tenant target blocked.
SLA timer repeats and device token is stale	One logical escalation per generation; expired device removed; notification failure does not erase inbox state.
Broker topic migration and cell failover	Key routing barrier preserves aggregate sequence; restored dedupe and quota records reconcile uncertain send windows.

Release evidence is attached to a versioned readiness record: test dataset and code version, measured SLOs, failure traces, provider adapter fixtures, recovery run, security findings, and named sign-off owners. Provider-only behaviors that cannot be simulated are demonstrated with approved test accounts and retained sanitized payloads. An unverified required provider contract blocks the corresponding production feature.

41 Enterprise implementation sequence

Keep the full enterprise architecture from the first production deployment. The sequence below orders engineering dependencies; it does not substitute a single-region MVP or postpone tenant isolation and recovery until after customers arrive. General availability follows completed evidence gates rather than an assumed 26-week calendar.

Workstream gate	Deliverable and exit evidence
A Platform foundation	Production/recovery accounts, three-zone cells, workload identity, network controls, secrets, CI, telemetry and automated cell provisioning; tested baseline restore.
B Messaging correctness	Onboarding and provider contracts, raw/routed/conversation streams, durable acknowledgment, outbox, quotas, send-state machine and independent receipts; failure-injection suite passes.
C Enterprise product operations	Agency grants/branding, imports/tags/segments, typed messages, campaigns, ad attribution, flow builder/runtime, agent tools and push; authorization and concurrency tests pass.
D Security and continuity	SSO/SCIM, audit, deletion, financial reconciliation, dedicated-cell placement, realistic capacity and regional failover; penetration findings resolved to approved thresholds.
E AI and launch readiness	Grounded retrieval, tool controls, evaluations, assisted replies and approved auto-send scopes; on-call, support, incident drills and customer acceptance complete.

Ownership and staffing model

Staff independent messaging, platform/SRE, customer-experience, automation/integration and security/data workstreams, with product/design and test automation embedded. Name owners for every service, provider integration and runbook. Regional coverage, leave, incident response and support queues must fit the actual staffing plan; two SREs alone do not provide a sustainable 24/7 commitment.

The earlier 12–16 person and roughly 26-week estimate is withdrawn as a delivery commitment. Estimate effort after provider approval discovery, the durable messaging slice, agency/branding requirements, representative load tests and the first recovery drill. Publish milestone ranges with dependencies, external approvals and confidence rather than a date implied by architecture alone.

Launch requirements

Launch requires all enterprise foundations and contracted product features above. AI can remain assisted where evaluations do not justify auto-send; this is a control decision, not an infrastructure downgrade. Roll out through internal tenants, approved pilot tenants and limited cells, with error-budget gates before expanding. New channels require their own capability matrices and contract tests.

42 Decisions to finalize before engineering

Decision	Recommended default	What changes the choice
Provider relationship	Official Cloud API with tenant-owned WABAs; direct provider billing where supported	Partner eligibility, billing responsibilities, customer account model.
Cloud and launch regions	One cloud; tenant home regions with approved recovery pairing	Residency, procurement, latency, service availability and contracts.
Tenancy	Shared bounded cells with dedicated-cell option	Customer isolation requirements, size and economics.
Regional writes	One active write region per tenant	A verified business need for global multi-writer state, with added complexity.
Data and execution	PostgreSQL + Kafka + durable workflows + object storage	Measured throughput and managed-service availability.
Message dispatch	One gateway; explicit unknown state after ambiguous send	A provider-supported idempotent send API may simplify recovery.
AI launch mode	Draft or handoff until business evaluations pass	Measured quality, action sensitivity, approved models and data policies.
Retention and recovery	Contract-driven policy with tested deletion and restores	Legal holds, regulated data, regional replication constraints.
Service promises	Publish only measured SLAs and support commitments	Load tests, recovery drills, on-call staffing and provider dependencies.

Engineering invariants

• Every tenant-owned read, write, event, cache entry, object, and external action has an enforced tenant boundary.

• An accepted webhook and an accepted send command each have durable evidence before acknowledgment.

• Every retryable action has a stable identity; every ambiguous external action has an explicit reconciliation path.

• Only the currently authorized owner can initiate a new reply, and only the outbound gateway can send it.

• Campaign throughput, AI usage, and customer integrations cannot exhaust reserved live-messaging capacity.

• A tenant has one active writer region; recovery never enables two independent senders.

• Published flow versions, policy decisions, and administrative changes remain traceable for the required retention period.

The enterprise launch includes the revised acknowledgment, quota, agency, import, attribution, rich-message and agent contracts. The product will feel seamless when these guarantees are visible through clear state: queued, sending, delivered, waiting for input, assigned to an agent, blocked by policy, or needing recovery. Hiding uncertainty creates duplicate messages and confusing support cases. Design the user interface around the actual distributed-system state.

Sources and implementation references

Research checked on 29 September 2026. Public product pages establish feature scope; official technical references support protocol and infrastructure choices. All service boundaries, capacity examples, SLOs, retention defaults, and rollout estimates are design proposals. The references below do not establish WhatChimp's internal technology stack.

1 WhatChimp product overview

https://whatchimp.com/

Public feature scope: campaigns, inbox, AI and integrations.

2 WhatChimp chatbot guide

https://help.whatchimp.com/docs/bot-manager/build-a-complete-whatsapp-chatbot-step-by-step

Public visual-flow behavior; reference for feature parity.

3 Meta official WhatsApp API workspace

https://www.postman.com/meta/whatsapp-business-platform/overview

Cloud API, Business Management, WhatsApp Flows and Embedded Signup overview.

4 Meta Cloud API collection overview

https://www.postman.com/meta/whatsapp-business-platform/collection/wlk6lh4/whatsapp-cloud-api

Account assets, permissions and access-token concepts. Confirm selected onboarding path.

5 Meta Messages API collection

https://www.postman.com/meta/whatsapp-business-platform/folder/13382743-ba8d099d-007e-4b52-b9f2-3cf3c60e4fbc

Provider message endpoint and supported message categories.

6 Meta webhook verification reference

https://whatsapp.github.io/WhatsApp-Nodejs-SDK/api-reference/webhooks/start/

GET challenge and POST signature behavior. The SDK is archived; use as protocol background, not a runtime dependency.

7 AWS transactional outbox pattern

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

Reliable publication of database state changes and the need for consumer idempotency.

8 Apache Kafka design documentation

https://kafka.apache.org/41/design/design/

Partition ordering, replication and producer durability concepts.

Further references and validation notes

9 WhatsApp Business Platform pricing

https://whatsappbusiness.com/products/platform-pricing/

Delivered-message pricing, categories, market dependence and customer-service-window explanation. Use current account rate cards at launch.

10 Temporal execution architecture

https://github.com/temporalio/temporal/blob/main/docs/architecture/README.md

Durable workflows, deterministic execution and idempotent activities. See also https://docs.temporal.io/tasks.

11 Kubernetes Network Policies

https://kubernetes.io/docs/concepts/services-networking/network-policies/

Network isolation, ingress and egress policies and implementation requirements.

12 OWASP SSRF prevention guidance

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

Destination restrictions and defensive URL-fetching controls.

13 PostgreSQL row security policies

https://www.postgresql.org/docs/18/ddl-rowsecurity.html

Policy enforcement and owner, superuser and BYPASSRLS exceptions.

Provider details to validate during implementation

Some Meta developer pages were login- or rate-limited during research. Official Meta-maintained API collections and the public WhatsApp pricing page provide the accessible foundation. Before implementing, confirm the current Graph API version, Embedded Signup eligibility and scopes, webhook event schemas and retry horizon, portfolio-level limits, per-number throughput, coexistence support, template rules, and WhatsApp Flows encryption requirements against the live developer portal.

Do not copy marketing claims about unlimited broadcasting, zero ban risk, or fixed approval times into service guarantees. Account restrictions and delivery remain subject to provider policy, account quality, recipient behavior, and available capacity. Price schedules can change independently of your application release, so the architecture keeps them effective-dated and configurable.

Glossary

CDC change data capture. DLQ dead-letter queue. HMAC keyed message authentication code. IAM identity and access management. mTLS mutual TLS. OIDC OpenID Connect. RLS row-level security. SCIM identity provisioning protocol. SLO service-level objective. SSRF server-side request forgery. WAF web application firewall. WSS WebSocket over TLS.

Additional provider and notification references

14 Meta webhook payload examples

https://www.postman.com/meta/whatsapp-business-platform/folder/vzaxn16/webhook-payload-reference

Official examples include status observations, CTWA-triggered messages, reactions, location and quick-reply callbacks. Some collection language is legacy; live fixtures remain required.

15 Firebase message ordering and collapse behavior

https://firebase.google.com/docs/cloud-messaging/customize-messages/collapsible-message-types

FCM does not guarantee delivery order; collapsible notifications can prompt a client resync. Provider behavior is distinct from durable inbox state.

16 Apple push notification requests

https://developer.apple.com/documentation/usernotifications/sending-notification-requests-to-apns

Official APNs integration reference for server requests and device targeting; implement current authentication and response handling.

17 Meta management webhooks and messaging limits

https://developers.facebook.com/documentation/business-messaging/whatsapp/webhooks/overview

Implementation verification entry point. Detailed pages returned access/rate-limit errors during review; candidate fields in the registry require live confirmation.

18 Meta business messaging conversions

https://developers.facebook.com/docs/marketing-api/conversions-api/business-messaging/

Implementation verification entry point for optional conversion export. Permissions, attribution field mapping and payload contract must be confirmed before enabling export.

Evidence boundaries

A source confirms only the behavior it describes. The architecture’s internal schemas, policy freshness, quota leases, region selection, timing targets and staffing gates are engineering decisions. No private WhatChimp source code or infrastructure was available. External subscription fields and pricing are not treated as immutable constants.

Before production activation, retain sanitized real webhook fixtures, successful subscription checks, verified asset ownership, effective capability observations and provider error classification for the exact app and API version. Unknown fields remain durable and visible for review. A missing required contract is an activation failure, not an assumed successful integration.